

Balanced Hypergraph Partitioning in the Hyperedge-Stream Model

Christian Schulz*

Abstract

Balanced k -way hypergraph partitioning is a key preprocessing step in VLSI design, parallel sparse matrix computations, SAT solving, and distributed data placement. High-quality multilevel partitioners require the entire hypergraph in internal memory, which is impossible for very large instances and unaffordable when a partition is needed in a tiny fraction of the time of the computation it enables. Streaming algorithms address both regimes, but current streaming hypergraph partitioners operate in the *vertex-stream* model, in which each vertex arrives together with the list of its incident nets. Since the de facto standard hMetis file format stores hypergraphs as hyperedge lists, these algorithms must first transpose the input on disk; this adds two preprocessing passes over the data and increases the storage footprint. We present HedgeStream, a streaming algorithm for the *hyperedge-stream* model that consumes hMetis-format inputs directly, in one sequential pass over the file per streaming pass. HedgeStream replaces exact per-vertex block affinities by fixed-capacity Misra–Gries sketches, so that its persistent state grows with the number of vertices plus an additive term in the number of blocks, independent of the number of nets and pins; only a first-pass buffer that holds the pins of a constant number of hyperedges temporarily adds to this. The per-vertex state is the smaller one once nets outnumber vertices several times over (between two- and five-fold at the constants of our implementation); below that, the per-net state of the vertex-stream tools stays smaller. The model is thus more than a different stream order: it shifts the memory dependence from the nets to the vertices. On 310 benchmark hypergraphs and six values of k , a single pass of HedgeStream improves connectivity over the state-of-the-art vertex-stream algorithm **FREIGHT** by a factor of 1.51 and cut-net by 1.15 in geometric mean, at about three times **FREIGHT**’s time per pin; on average it also outperforms two in-memory partitioners at a fraction of their time and memory: **HYPE** on both objectives, and **SIGHP** in connectivity, driven by the sparse-matrix family.

1 Introduction

Hypergraphs generalize graphs by allowing a single (hyper)edge to connect any number of vertices, and are therefore the natural model wherever relations are multi-way rather than pairwise: a signal net of a VLSI circuit connects all cells it touches, a clause of a SAT formula connects all variables it mentions, and a row of a sparse matrix connects all columns in which it has a nonzero. Balanced k -way hypergraph partitioning, i.e. dividing the vertices into k blocks of nearly equal size while minimizing the number of blocks spanned by the hyperedges, is a core preprocessing step in all of these domains: it captures the communication volume of parallel sparse matrix–vector

*Heidelberg University, Heidelberg, Germany

multiplication exactly [11], drives cut-based physical design flows [28], and governs data placement in distributed databases and storage systems. The problem is NP-hard, and decades of algorithm engineering have produced multilevel partitioners such as **hMetis** [28], **PaToH** [11], and **KaHyPar** [40] that compute partitions of very high quality. These tools target the offline setting, however: they keep the entire hypergraph, plus a multilevel hierarchy built on top of it, in internal memory, and they invest substantial running time to earn their quality. This is a poor fit for two situations that are becoming ever more common. The first is sheer size: when a hypergraph does not fit into the memory of the machine that has to partition it, no amount of tuning helps, and one has to give up the global view of the instance altogether. The second is the opposite regime, in which the instance fits comfortably but the time budget is essentially zero. Distributed computing frameworks and databases still shard their data by hashing simply because hashing is free, loaders of parallel applications have to place data before the actual computation may start, and even heavyweight partitioners profit from an informed initial distribution of the input across machines. What these settings need is an algorithm that costs little more than a hash function, yet returns partitions that are dramatically better than random assignment.

Streaming algorithms answer both needs with the same mechanism: the input is read sequentially, every element is placed immediately, or after a bounded delay in the buffered variants, based on a small working state, and only the resulting partition is kept. For graphs, this line of work is well developed, from the seminal one-pass heuristics of Stanton and Kliot [43] over the **Fennel** objective of Tsourakakis et al. [45] to restreaming [34] and buffered streaming [5]. For hypergraphs, early work targeted min-max load objectives [2], and **FREIGHT** [17] is the current state of the art for the balanced connectivity and cut-net objectives. Like its graph ancestors, **FREIGHT** operates in the *vertex-stream* model: vertices arrive one by one, each together with the IDs of its incident nets, and are assigned to a block on arrival, while the algorithm keeps state for every net seen so far, $\Theta(m)$ words in total.

Lastly, a frequently used format to store hypergraphs is **hMetis** [28], a *hyperedge list* in which line i holds the pins of net e_i . A vertex-stream partitioner cannot read it and must first *transpose* it into vertex-to-net lists. The transposition does stream, in one pass that counts vertex degrees to fix the offset of every net list and a second that writes the pins into place, but it is not free: it touches the data three times before the partitioner sees a single vertex, and the partitioner then reads the copy once per streaming pass.

Contribution. *First*, we present **HedgeStream**, to the best of our knowledge the first balanced partitioner for the *hyperedge-stream* model, in which hyperedges arrive one at a time with their complete pin lists, exactly the order and shape in which the **hMetis** format stores them. Its idea is simple: each arriving hyperedge greedily places its still unassigned pins and updates, for every pin, an estimate of how strongly that vertex is attracted to each block, and these accumulated affinities then drive local moves. *Second*, we cut the memory this needs. Kept exactly, the affinities would be an $O(n \cdot k)$ array of counters, for large k larger than the hypergraph itself, so we retain only an estimate of the heavy hitters of each vertex’s distribution in a fixed-capacity Misra–Gries sketch of a few slots, which leaves no per-net state behind once a hyperedge has been processed. This is the smaller state once nets outnumber vertices several times over, between two- and five-fold at the measured constants (Section 3.6), as on many SAT formulas and sparse matrices; below that, the per-net state of the vertex-stream tools remains smaller, and Section 4.6 measures both regimes. *Third*, we recover the quality that this approximation costs, through prioritized buffering with an incremental bucket-queue flush that always processes the currently best-informed hyperedge next, a fused move evaluator that combines the sketched long-term affinity with exact edge-local pin counts, a final sweep that re-evaluates every vertex against its completed sketch, and restreaming in which

the sketches accumulate across passes while hyperedges that have become internal are skipped. *Fourth*, we engineer for speed: a candidate restriction bounds the work per pin independently of k and, in the component study, saves 23% of the running time while improving connectivity by 4.7%, and large-net rules save up to a factor of 2.3 on dense instances.

A single truly sequential pass over the unmodified input file therefore suffices, with no conversion and no copy. The price is that a vertex is never seen together with all of its nets, which is what deciding its block would require; Section 3.1 shows how the affinities that the sketches accumulate stand in for the missing neighborhood.

2 Preliminaries

2.1 Basic Concepts

Hypergraphs and Graphs. Let $H = (V = \{1, \dots, n\}, E)$ be an *undirected hypergraph* with no multiple hyperedges and no hyperedges of size at most one (the implementation drops such nets on input), with $n = |V|$ vertices and $m = |E|$ hyperedges (or *nets*). A *net* is defined as a subset of V . The vertices that compose a net are called *pins*. A vertex $v \in V$ is *incident* to a net $e \in E$ if $v \in e$. Let $c : V \rightarrow \mathbb{R}_{\geq 0}$ be a vertex-weight function, and let $\omega : E \rightarrow \mathbb{R}_{> 0}$ be a net-weight function. We generalize c and ω functions to sets, such that $c(V') = \sum_{v \in V'} c(v)$ and $\omega(E') = \sum_{e \in E'} \omega(e)$. Let $I(v)$ be the set of incident nets of v and let $d(v) := |I(v)|$ be the *degree* of v . Let the number of pins $|e|$ in a net e be the *size* of e . The *pin-count* $P = \sum_{e \in E} |e|$ of a hypergraph is the total number of pins in all of its nets.

Reusing the notation, let $G = (V = \{1, \dots, n\}, E)$ be an *undirected graph* with no multiple or self edges, such that $n = |V|$, $m = |E|$. Let $N(v) = \{u : \{v, u\} \in E\}$ denote the neighbors of v , and let $d(v)$ be the degree of vertex v ; vertex and edge weights c and ω are defined as for hypergraphs.

Partitioning. The *(hyper)graph partitioning* problem consists of assigning each vertex of a (hyper)graph to exactly one of k distinct *blocks* respecting a balancing constraint in order to minimize the weight of the (hyper)edges running between the blocks, i.e., the edge-cut (resp. cut-net). More precisely, it partitions V into k blocks $V_1, \dots, V_k$ (i.e., $V_1 \cup \dots \cup V_k = V$ and $V_i \cap V_j = \emptyset$ for $i \neq j$), which is called a *k-partition* of the (hyper)graph. The *edge-cut* (resp. *cut-net*) of a *k-partition* is the total weight $\omega(E')$ of the *cut edges* (resp. *cut nets*), those crossing blocks: $E' := \{e \in E : \exists i \neq j, \exists u, v \in e : u \in V_i, v \in V_j\}$ is the *cut-set*. The *balancing constraint* demands that the sum of vertex weights in each block does not exceed a threshold $L_{\max}$ determined by the permitted imbalance ε . More specifically, $\forall i \in \{1, \dots, k\} : c(V_i) \leq L_{\max} := \lceil (1 + \varepsilon) \frac{c(V)}{k} \rceil$. For each net e of a hypergraph, $\Lambda(e) := \{V_i \mid V_i \cap e \neq \emptyset\}$ denotes the *connectivity set* of e . The *connectivity* $\lambda(e)$ of a net e is the cardinality of its connectivity set, i.e., $\lambda(e) := |\Lambda(e)|$. The so-called *connectivity* metric is computed as $\sum_{e \in E'} (\lambda(e) - 1) \omega(e)$, where E' is the cut-set. Due to its mathematical definition, the connectivity metric is also called the λ -1 metric.

Streaming. Streaming algorithms usually follow an iterative load-compute-store logic. The most used streaming model is the one-pass *vertex-stream* model, in which the vertices of a hypergraph are loaded one at a time alongside with their incident nets, and then some logic is applied to permanently assign them to blocks. This logic can be as simple as a **Hashing** function or as complex as scoring all blocks based on some objective and then assigning the vertex to the block with the highest score. Note that a vertex is loaded together with the identifiers of its incident nets, but

without the pins of those nets. In this work, we use a different variant, the *hyperedge-stream* model, in which the nets of a hypergraph are loaded one at a time alongside with their complete pin lists, and each of them updates the state of the algorithm before the next one arrives; a buffered variant may defer a bounded number of nets and process them later in a different order (Section 3.2). For graphs, a streaming algorithm is commonly held to a memory capacity of $O(n + k)$; for hypergraphs, prior streaming algorithms [2, 44] use up to $O(mk)$ memory, and no strict budget has been established; HedgeStream adopts a budget of $O(n \cdot t + k)$ persistent words, independent of m and smaller than the $\Theta(m)$ -word per-net state of the vertex-stream tools once nets outnumber vertices several times over, plus a buffer of at most $B = 2^{14}$ hyperedges (Section 3.6). In both models we assume that n and m are known in advance; for hMetis inputs they are read from the file header.

2.2 Related Work

There is a huge body of research on (hyper)graph partitioning. The most prominent tools to partition (hyper)graphs in memory include PaToH [11], Metis [27], hMetis [28], Scotch [36], HYPE [29], KaHIP [39], KaMinPar [22], KaHyPar [40], Mt-KaHyPar [21], and mt-KaHIP [1]. A recent single-threaded in-memory partitioner for very large hypergraphs is SIGHP [23], which guides its greedy assignment by an information-theoretic score, shields hub nets, and reports large gains over FREIGHT on its own benchmark; it reads the whole hypergraph into memory and is therefore a reference point rather than a competitor for the streaming setting, and we include it in Section 4. The readers are referred to [12, 10, 42] for extensive material and references. Here, we focus on the results specifically related to the scope of this paper. In particular, we provide a detailed review for the following problems in the streaming model: hypergraph partitioning and graph vertex partitioning.

Streaming Hypergraph Partitioning. Alistarh et al. [2] propose Min-Max, a one-pass streaming algorithm to assign the vertices of a hypergraph to blocks. For each block, this algorithm keeps track of nets which contain pins in it. This implies a memory consumption of $O(mk)$. When a vertex is loaded, Min-Max allocates it to the block containing the largest intersection with its nets while respecting a hard constraint for load balance. The authors theoretically prove that their algorithm is able to recover a hidden *co-clustering* with high probability, where a co-clustering is defined as a simultaneous clustering of vertices and hyperedges. In the experimental evaluation, Min-Max attains a lower load imbalance than five intuitive streaming approaches, but still one up to five times higher than that of internal-memory algorithms such as hMetis. Min-Max optimizes load balance alone. Both FREIGHT and HedgeStream instead optimize a quality metric (connectivity or cut-net) jointly with load balance, an approach proven effective in streaming graph partitioning [43, 45], at lower time and memory cost than Min-Max.

Taşyaran et al. [44] propose improved versions of the algorithm Min-Max [2]. The authors present Min-Max-N2P, a modified version of Min-Max that stores blocks containing each net’s pins instead of storing nets per block, as done in Min-Max. In their experiments, Min-Max-N2P is three orders of magnitude faster than Min-Max while keeping the same cut-net. The authors also introduce three algorithms with reduced memory usage compared to Min-Max: Min-Max-L ℓ , a modification of Min-Max-N2P that employs an upper-bound ℓ to limit memory consumption per net, Min-Max-BF which utilizes Bloom filters for membership queries, and Min-Max-MH that uses hashing functions to replace the connectivity information between blocks and nets. In their experiments, their three algorithms reduce the running time in comparison to Min-Max, especially Min-Max-L ℓ and Min-Max-MH, which are up to four orders of magnitude faster. On the other hand, the three algorithms generate solutions with worse cut-net than Min-Max, especially Min-Max-MH,

which increases the cut-net by up to an order of magnitude. Moreover, the authors propose a technique to improve the partitioning decision in the streaming setting by including a buffer to store some vertices and their net sets. This approach operates similarly to **Min-Max-N2P**, but with the added ability to revisit buffered vertices and adjust their partition assignment based on the connectivity metric. The authors propose three algorithms using this buffered approach: **REF** that buffers every incoming vertex but only reassigns those that may improve connectivity, **REF_RLX** that buffers all vertices and reassigns all vertices in the buffer, and **REF_RLX_SV** that only buffers vertices with small net sets and reassigns all vertices in the buffer. Their experimental results show that the use of buffered approaches leads to a 5-20% improvement in partitioning quality compared to non-buffered approaches, but with a trade-off of increased runtime. All of these variants process a vertex stream and keep per-net state. Their buffered algorithms (**REF** and its relaxations) are the closest precursors of our prioritized buffering (Section 3.2), which likewise defers poorly informed decisions, but keys the buffer by the assigned-pin fraction of a hyperedge and reorders it incrementally during the flush. These variants are also the closest prior use of approximate summaries in streaming hypergraph partitioning. They summarize the *net-to-block* relation and do so with membership rather than frequency structures: **Min-Max-BF** tests membership with Bloom filters, **Min-Max-MH** replaces the relation by hashing, and **Min-Max-L ℓ** truncates each net’s block list to ℓ entries. **HedgeStream** instead summarizes the *vertex-to-block* affinity, which is a weighted frequency distribution rather than a set, and keeps its heavy hitters in a fixed-capacity Misra–Gries summary. The difference to truncation is which entries survive: **Min-Max-L ℓ** keeps an arbitrary ℓ of them, whereas the summary is designed to keep the blocks that carry most of a vertex’s affinity, which is what the move evaluator of Section 3.1 uses, and that section states what the weighted variant does and does not guarantee.

Fernandez Musoles et al. [33] propose **HyperPRAW**, an architecture-aware hypergraph partitioner that restreams the input to improve the placement of parallel applications on a machine topology. It is the only earlier work that restreams hypergraphs, and the closest predecessor of our Section 3.4, but it also streams vertices with their net lists and targets communication bandwidth rather than connectivity or cut-net.

FREIGHT [17], the current state of the art and our main baseline, transfers the **Fennel** objective to hypergraphs in the vertex-stream model: each arriving vertex is assigned to the block maximizing an additive combination of attraction and balance penalty, where the attraction is computed from per-net state recording, for every net seen so far, the block of its most recently assigned pin together with a cut flag. This state is $\Theta(m)$ words and persists for the whole run; **FREIGHT** runs in time linear in the input size and optimizes connectivity or cut-net.

Streams in which the hyperedges rather than the vertices arrive are established for other hypergraph problems, among them semi-streaming hypergraph matching [38] and triangle counting [31]. To the best of our knowledge no partitioner has so far been formulated in this model.

Streaming Graph Vertex Partitioning. Stanton and Kliot [43] introduced graph partitioning in the streaming model and proposed some heuristics to solve it. Their most prominent heuristics include the one-pass methods **Hashing** and *linear deterministic greedy* (**LDG**). In their experiments, **LDG** had the best overall edge-cut. In this algorithm, vertex assignments prioritize blocks containing more neighbors and use a penalty multiplier to control imbalance. Particularly, a vertex v is assigned to the block V_i that maximizes $|V_i \cap N(v)| \cdot \psi(i)$ with $\psi(i)$ being a multiplicative penalty defined as $(1 - \frac{|V_i|}{L_{\max}})$. The intuition is that the penalty avoids to overload blocks that are already very heavy. In case of ties on the objective function, **LDG** assigns the vertex to the block with fewer vertices. Overall, **LDG** partitions a graph in $O(m + nk)$ time. On the other hand,

Hashing has running time $O(n)$ but produces a poor edge-cut. Like LDG, HedgeStream optimizes cut quality and load balance simultaneously, but, following **Fennel** and **FREIGHT**, with an additive rather than a multiplicative penalty.

Tsourakakis et al. [45] proposed **Fennel**, a one-pass partitioning heuristic based on the widely-known clustering objective *modularity* [9]. **Fennel** assigns a vertex v to a block V_i , respecting a balancing threshold, in order to maximize an expression of type $|V_i \cap N(v)| - f(|V_i|)$, i.e., with an additive penalty. This expression is an interpolation of two properties: attraction to blocks with many neighbors and repulsion from blocks with many non-neighbors. When $f(|V_i|)$ is a constant, the expression coincides with the first property. If $f(|V_i|) = |V_i|$, the expression coincides with the second property. In particular, the authors defined the **Fennel** objective with $f(|V_i|) = \alpha \cdot \gamma \cdot |V_i|^{\gamma-1}$, in which γ is a free parameter and $\alpha = m^{\frac{k\gamma-1}{n\gamma}}$. After a parameter tuning made by the authors, **Fennel** uses $\gamma = \frac{3}{2}$, which provides $\alpha = \sqrt{k} \frac{m}{n^{3/2}}$. As LDG, **Fennel** partitions a graph in $O(m + nk)$ time. **FREIGHT** extends the **Fennel** objective to hypergraphs; HedgeStream adopts the same objective for the assignment of new pins (Section 3.1), evaluated on hyperedge arrival rather than on vertex arrival.

Faraj and Schulz [19] propose a shared-memory streaming algorithm for vertex partitioning which performs recursive multi-sections on the fly. As a preliminary phase, their algorithm decomposes a k -way partitioning problem into a hierarchy containing $\lceil \log_b k \rceil$ layers of b -way partitioning subproblems. This hierarchy can either reflect the topology of a high performance system to solve a process mapping [20, 37] or be computed for an arbitrary k to solve a regular vertex partitioning. Then, an adapted version of **Fennel** is used to solve each of the subproblems in such a way that the whole k -partition is computed on the fly during a single pass over the graph. While producing an edge-cut around 5% worse than **Fennel**, their algorithm has theoretical complexity $O((m + nb) \log_b k)$ and experimentally ran up to two orders of magnitude faster than **Fennel**. HedgeStream computes a flat k -partition and uses no recursive multi-section.

Besides the one-pass model, other streaming models have also been used to solve vertex partitioning. Nishimura and Ugander [34] introduce a restreaming approach to partition the vertices of a graph. Their approach is motivated by scenarios where the same graph is streamed multiple times. In their model, a one-pass partitioning algorithm can pass multiple times through the entire input while the edge-cut is iteratively reduced. The authors propose **ReLDG** and **ReFennel**, which are respective restreaming adaptations of linear deterministic greedy [43] (LDG) and **Fennel** [45]. On the one hand, **ReLDG** modifies the objective of LDG to account only for vertex assignments performed during the current pass when computing block weights. On the other hand, **ReFennel** uses the same objective as **Fennel** during restreaming, but its additive balancing degrading factor is increased after each pass in order to enforce balance. Additionally, the authors prove that **ReFennel** converges after a finite number of restreams even without increasing the degrading factor. Their experiments confirm that their restreaming methods can iteratively reduce edge-cut. HedgeStream operates in this model as well: Section 3.4 restreams the hyperedge stream and follows **ReFennel** in raising the balance penalty from pass to pass.

Awadelkarim and Ugander [4] investigate how the order in which vertices are streamed influences one-pass graph partitioning. The authors introduce the notion of *prioritized streaming*, where (re)streamed vertices are statically or dynamically reordered based on some predefined priority. Their approach, which is a prioritized version of **ReLDG**, uses multiplicative weights of restreaming algorithms and adapts the ordering of the streaming process inspired by balanced label propagation. In their experiments, the authors consider a wide range of stream orderings. The minimum overall edge-cut is obtained using a dynamic vertex ordering based on their own metric *ambivalence*. This approach is closely followed by a static ordering based on vertex degree. HedgeStream

transfers this idea to the hyperedge-stream model: the buffered flush of Section 3.2 dynamically reorders buffered hyperedges by the fraction of their pins already assigned, a hyperedge analogue of prioritized streaming.

An extended streaming model is the *buffered* streaming model. During execution, a *buffer* or *batch* of input vertices and their neighborhoods are repeatedly loaded, or a sliding window is maintained in memory. Patwary et al. [35] propose **WStream**, a simple streaming graph partitioning algorithm that keeps a sliding window in memory. The authors allow a few hundred vertices in the sliding window in order to obtain more information about a vertex before it is permanently assigned to a block based on a greedy function. As soon as a vertex is allocated to a block, one more vertex is loaded from the input stream into the sliding window, which keeps the window size constant. In their experiments, **WStream** cuts fewer edges than LDG and more edges than offline multilevel partitioning for most tested graphs. The streaming model and the objective function of **WStream** both differ from ours.

Jafari et al. [26] perform graph partitioning using a buffered streaming computational model. The authors propose a shared-memory algorithm which repeatedly loads a batch of vertices from the stream input, partitions it using a multilevel scheme, and then permanently assigns the vertices to blocks. Their multilevel scheme is based on a simplified structure where the one-pass algorithm LDG is used for coarsening, computing an initial partition, and refining it. They parallelize LDG in a vertex-centric way by simply splitting vertices among processors, which yields a parallelization of the three steps of their multilevel scheme. In their experiments, their algorithm cuts fewer edges than LDG while scaling better than offline partitioning algorithms. HedgeStream likewise buffers part of the input, but processes the buffer with the streaming rule itself rather than a multilevel scheme.

Faraj and Schulz [18] propose **HeiStream**, an algorithm which also partitions vertices in a buffered streaming model. Their algorithm loads a batch of vertices, builds a graph model, and then partitions this model with a multilevel algorithm. In their graph model, the vertices from previous batches assigned to each block are represented as a single big vertex fixed to the respective block. Analogously, an edge between a vertex v from the current batch b and a vertex $\bar{v}$ from a previous batch $\bar{b}$ is represented by an edge $(\bar{b}, b)$. In addition, when a vertex from a current batch has a neighbor from a future batch (i.e., not yet streamed), their model compactly represents this neighbor in a contracted form. Their multilevel algorithm has a traditional structure and components, except that the initial partitioning is a one-pass execution of **Fennel** and the label propagation refinement also optimizes the objective function used by **Fennel** to assign vertices to blocks. In particular, the **Fennel** objective is extended to weighted graphs. In experiments, **HeiStream** cuts fewer edges than LDG, **Fennel**, and the buffered streaming algorithm proposed by Jafari et al. [26], while being faster than **Fennel** for large numbers of blocks. Like **HeiStream**, HedgeStream extends the **Fennel** objective and buffers part of the stream, but it partitions hypergraphs and keeps only sketched per-vertex state instead of building a graph model of the buffer.

Buffered streaming has since been developed further. Hajidehi et al. [24] propose **CUTTANA**, which buffers vertices and prioritizes which of them to assign next, so that premature decisions are deferred; this is the closest prior art for the prioritized buffering of Section 3.2, which orders hyperedges instead of vertices and maintains the order incrementally during the flush. Hoffmann et al. [25] extend buffered streaming to vertex- and edge-balanced partitioning for distributed training, and Baumgartner et al. [5] apply it to the cut-based objective.

Streaming Edge Partitioning. A parallel line of work partitions the *edges* of a graph rather than its vertices, and therefore also consumes a stream of edges rather than of vertices.

Algorithm 1 HedgeStream, the basic idea (details, including the restriction of the sketch update to a candidate set and the buffering of Section 3.2, in Algorithms 2 and 3)

Require: hyperedge stream, k, ε ; per vertex a block and a sketch of t (block, score) slots

- 1: **for each** hyperedge e in the stream **do**
 - 2: count the pins of e per block
 - 3: place each unassigned pin of e in the block with most pins of e , minus a load penalty
 - 4: **for each** pin v of e and block b holding other pins of e **do**
 - 5: add the share of e 's pins in b to v 's sketch slot for b ▷ Misra–Gries, t slots
 - 6: **for each** pin v of e **do**
 - 7: move v to the block its sketch and e favor, if that beats staying
 and does not split e by the counts at hand ▷ Eq. (4)
 - 8: sweep over all vertices, moving each to the best block of its complete sketch, until few move
-

Mayer et al. [30] give an out-of-core edge partitioner running in linear time, Ding et al. [16, 15] phrase the problem as a game over streamed clusters, Bhoot et al. [8] preserve community structure under arbitrary stream orders, and Chhabra et al. [13] propose a buffered streaming algorithm and a FREIGHT-based one-pass algorithm for assigning the edges of a graph to blocks; neither streams hyperedges. The models are superficially similar to ours, since both read a stream of (hyper)edges, but the output differs: these algorithms assign the arriving edges to blocks, whereas HedgeStream assigns the *vertices* and reads hyperedges only as evidence about where they belong.

3 HedgeStream

Throughout this section we assume unit vertex and net weights, so that $c(V_i) = |V_i|$ and the cut-net metric counts hyperedges; the algorithm extends to weights by replacing counts with weighted sums; the evaluation of Section 4 covers the unit-weight case only. We first develop the basic algorithm, which follows directly from the constraints of a hyperedge stream and is deliberately kept simple (Section 3.1). We then identify concrete weaknesses of that algorithm and address them in turn: the first hyperedges of a stream are decided with no information at all (Section 3.2), a vertex's last decision within a pass is taken before its sketch is complete (Section 3.3), and a single pass leaves quality on the table (Section 3.4). Sections 3.5 and 3.6 cover the cut-net objective, the memory layout, and the resources of the resulting algorithm. The parameters t, θ, B , the number of sweep rounds and the number of passes were fixed in advance on the tuning instances of Section 4.1, which are part of the benchmark, and Section 4.1 reports their effect; the remaining constants were set by hand or in preliminary experiments on the same instances and are not varied; Table 2 collects every constant, its value and how it was fixed, so that the degrees of freedom of the design can be audited. Algorithm 1 states the basic idea in one listing before any engineering: a hyperedge arrives, its unassigned pins are placed by a load-aware greedy rule, every pin records in a small per-vertex sketch which blocks its nets keep pointing to, and every pin may then move to the block its sketch favors; a final sweep re-evaluates every vertex once its sketch is complete; buffering hyperedges that arrive with too little context and flushing them in a best-informed-first order extends this idea in Section 3.2.

Algorithm 2 HedgeStream: processing one hyperedge (the home score $s(v, \pi(v))$ includes $\text{credit}(v)$; $\bar{s}(v)$ is the upper bound on $\max_{b \neq \pi(v)} s(v, b)$ of Section 3.1)

Require: hyperedge e (next from the stream, or dequeued from the buffer during a flush), partition π , block sizes $|V_1|, \dots, |V_k|$

- 1: **if** pass 0 **and** fewer than a θ fraction of e 's pins are assigned **then**
- 2: buffer e and **return** ▷ Section 3.2
- ▷ flushed once it holds B nets, and at the end of the pass: each buffered hyperedge,
 ▷ best-informed first, is then processed by the lines below, skipping this test
- 3: $new \leftarrow \emptyset$; $c \leftarrow \mathbf{0}$ ▷ sparse: only occupied blocks stored
- 4: **for** $v \in e$ **do** ▷ Phase 1: classify and count, one pass
- 5: $\text{seen}(v) \leftarrow \text{seen}(v) + 1$
- 6: **if** $\pi(v) = \perp$ **then** $new \leftarrow new \cup \{v\}$
- 7: **else** $c_{\pi(v)} \leftarrow c_{\pi(v)} + 1$
- 8: **if** $new = \emptyset$ **and** exactly one block is occupied **then**
- 9: $\text{credit}(v) \leftarrow \text{credit}(v) + 1$ for all $v \in e$; **return** ▷ e is internal: credit and skip
- 10: $\rho_b \leftarrow \alpha \gamma |V_b|^{\gamma-1}$ for every occupied b and the least-loaded block ▷ from the penalty table
- 11: **for** $v \in new$ **do** ▷ Phase 2: assign
- 12: $b^* \leftarrow \arg \max \{c_b - \rho_b : |V_b| < \lceil n/k \rceil\}$ over occupied $\cup$ least-loaded
- 13: $\pi(v) \leftarrow b^*$; $|V_{b^*}| \leftarrow |V_{b^*}| + 1$; $c_{b^*} \leftarrow c_{b^*} + 1$
- 14: $\mathcal{C} \leftarrow$ two occupied blocks with most pins, plus ties, at most 8 ▷ all if ≤ 2 occupied
 plurality block only if $|e| > 128$
- 15: **for** $v \in e$ **do** ▷ Phases 3 and 4, fused
- 16: **for** $b \in \mathcal{C}$ with $c_b - [\pi(v)=b] \geq c_{\min}$ **do** ▷ $c_{\min} = 2$ if more than 8 blocks are occupied, else 1
- 17: $\text{UPDATE}(v, b, w(v, b))$ ▷ Alg. 3; w_{cut} in cut-net mode
- 18: **if** $\text{seen}(v) \geq 2$ **and** v tracks ≥ 2 blocks **and** $\bar{s}(v) > s(v, \pi(v))$ **then**
- 19: $T \leftarrow$ blocks tracked by v , plus $\mathcal{C}$ if $\text{seen}(v) > 5$
- 20: $b^* \leftarrow \arg \max \{s(v, b) : b \in T, b \neq \pi(v), |V_b| < L_{\max}\}$
- 21: **if** $s(v, b^*) > s(v, \pi(v))$ **and** ($c_{\pi(v)} \leq 1$ **or** $c_{b^*} > 0$) **then** ▷ the move cannot raise $\lambda(e)$
- 22: fold $\text{credit}(v)$ into v 's sketch (rule of Phase 3); $\text{credit}(v) \leftarrow 0$;
 $|V_{\pi(v)}| -= 1$; $|V_{b^*}| += 1$; $\pi(v) \leftarrow b^*$

3.1 The Basic Algorithm

A hyperedge arrives as a set of pins, and the algorithm must react to it before the next one arrives. Two decisions have to be made. First, the pins of e that have never been seen before need a block, and that decision cannot be deferred: it must be made from whatever the algorithm knows at this moment. Second, the pins of e that already carry an assignment may have been placed badly, because when they were placed only a fraction of their nets had streamed past; the arrival of e is new evidence, and it is the only occasion on which these vertices can be reconsidered before they disappear from view again. The basic algorithm therefore processes each hyperedge in four phases: it summarizes where the pins of e currently live, assigns the new ones, updates a per-vertex record of block affinity, and re-evaluates the placement of every pin. Algorithm 2 states the resulting procedure.

Phase 1: classification and sparse counting. Everything the later phases need about e is captured by a single quantity, namely how many of e 's pins already sit in each block: a block that already holds many pins of e is an attractive home for the remaining ones, because putting them there keeps e from spanning yet another block. We therefore classify each pin of $e = \{v_1, \dots, v_p\}$ as *new* (not yet assigned) or *old*, and maintain the *co-pin count*

$$c_b = |\{v \in e : \pi(v) = b\}|, \quad (1)$$

where π is the current partition and $\pi(v) = \perp$ marks an unassigned vertex. (The letter c hereafter denotes these counts; the vertex-weight function of Section 2 does not recur, block loads being written $|V_b|$ under the unit-weight assumption above.) A dense k -entry vector would make every hyperedge cost $\Omega(k)$ regardless of its size, which is prohibitive for large k on instances whose nets have only a handful of pins. We therefore store c sparsely and record only the *occupied* blocks, those with $c_b > 0$, of which there are at most $q \leq \min(p, k)$. Classification and counting are fused into one loop over the pins, so the phase costs $O(p)$ time and touches each pin's vertex record exactly once. The same loop increments a per-vertex counter $\text{seen}(v)$ of how many hyperedges of the current pass have contained v (a per-pass count, not the degree $d(v)$ of Section 2); Phase 4 uses it to decide whether v has been seen often enough to be worth reconsidering; a buffered hyperedge increments it when it is flushed.

If all pins are old and occupy a single block, the hyperedge is internal, contributes $\lambda(e) - 1 = 0$, and cannot be improved. We return immediately, which skips Phases 2–4 for such hyperedges. This *single-block skip* is a heuristic trade rather than a free lunch: it also forgoes the moves that Phases 3 and 4 might have made for e 's pins on the strength of *other* nets, but those moves would have to leave e internal to be worthwhile, and the skip is what makes internal hyperedges essentially free.

Phase 2: assignment of the new pins. Each new vertex $v \in e$ is placed greedily. Following Fennel [45], we trade the attraction of a block against its current load,

$$b^*(v) = \arg \max_{b: |V_b| < L_{\max}} \left[\underbrace{c_b}_{\text{attraction}} - \underbrace{\alpha \gamma |V_b|^{\gamma-1}}_{\text{balance penalty } \rho_b} \right], \quad (2)$$

with $\gamma = 3/2$ and the capacity bound $L_{\max}$ of Section 2, for which the next sentence substitutes a tighter cap; m is the number of nets. For the placement of new vertices we replace $L_{\max}$ in (2) by the tighter cap $\lceil n/k \rceil$, which reserves the ε slack for the moves of Phase 4 and the sweep of Section 3.3: without the cap, on inputs whose nets have many pins, the first pass fills every block to $L_{\max}$, because a net with many new pins places them all into its plurality block, whose attraction c_b grows with every placement while the cached penalty ρ_b does not change within the net, and no later move has a target with room. The least-loaded block is found in constant time from a block order sorted by size that is maintained by boundary swaps on every size change, similar to the self-sorting array used in FREIGHT [17]. Fennel tunes $\alpha = \sqrt{k} m / n^\gamma$ for graphs; we scale it to $\alpha = 0.7 \sqrt{k} m / n^\gamma$, a factor fixed in preliminary experiments on the tuning instances; the direction is what one expects, since a hyperedge contributes attraction from up to $|e| - 1$ co-pins at once, so the untempered penalty is comparatively too strong. The block cost $\alpha |V_b|^\gamma$ behind the penalty is deliberately super-linear in the block size, so that its marginal cost ρ_b grows with $|V_b|$: with $\gamma > 1$ an already heavy block is punished disproportionately, which keeps the partition balanced without relying on the hard cap alone, since a cap that is reached forces arbitrary placements. Two details keep the phase cheap. The penalties ρ_b depend only on the block sizes, so we tabulate $x \mapsto x^{\gamma-1}$ for all $x \leq L_{\max}$ once at construction time, which replaces a transcendental function call

per candidate by an array lookup, and we cache ρ_b for the occupied blocks once per hyperedge. Moreover, we restrict the $\arg \max$ to the occupied blocks together with the least-loaded block, so that a hyperedge never pays $\Theta(k)$; the least-loaded block is always below $\lceil n/k \rceil$, since fewer than n vertices are placed at any time, so the $\arg \max$ is never empty. Every assignment updates $|V_b|$ and c_b immediately, so a new pin becomes visible as a co-pin to the pins processed after it, both in the remainder of Phase 2 and in Phases 3 and 4. A vertex that occurs in no hyperedge is never seen by the stream; it is placed in the lightest block when the partition is written, and it counts toward n in $L_{\max}$.

Phase 3: affinity as a substitute for the missing neighborhood. Phase 2 places a vertex using the evidence of a single hyperedge. This is necessarily short-sighted: the other nets of v arrive later, and when they do they may reveal that most of v 's co-pins live somewhere else entirely. Correcting the placement at that point requires knowing how v 's nets are spread over the blocks, but those nets have already streamed past and are not stored. We therefore give every vertex a memory of what it has seen. The *affinity* $a(v, b)$ accumulates, over all hyperedges processed so far, how strongly v is pulled toward block b ; when hyperedge e arrives, every pin $v \in e$ and every block b of the candidate set $\mathcal{C}$ defined below contribute

$$w(v, b) = \frac{c_b - [\pi(v) = b]}{|e| - 1}. \quad (3)$$

The indicator removes v 's own contribution, since v is not evidence for where v already is, and updates with $w(v, b) = 0$ are suppressed, so that a block never enters a sketch on the strength of no evidence. The normalization is what makes affinities comparable across nets of very different size. Without it, a net with 1000 pins would push three orders of magnitude more weight into the sketch than a net with two pins, although being one pin among a thousand is much *weaker* evidence of belonging together than being one of two. Dividing by $|e| - 1$ caps the total contribution of any net to one unit per pin, so that every net gets one vote.

Two refinements concern the nets at the extremes. An internal net is the strongest evidence for staying, yet the single-block shortcut of Phase 1 discards its unit update (3). We keep it as *home credit*, a per-vertex counter stored next to the partition (Section 3.6) and incremented once per internal net. The credit is added to the home block's score in every move evaluation. The credit is never decremented, so a net that was internal when credited and is cut later leaves its credit in place. When the vertex moves, the credit is folded into the sketch: into the old block's slot if it is tracked, otherwise into a free slot, otherwise into the weakest slot, whose block becomes the old block and whose score becomes the credit, if the credit exceeds that score, and it is discarded if not; in all cases without a decrement, and the counter is cleared. Without it the sketch sees only cut nets and the move rule pulls vertices out of perfectly internal clusters. The credit is exact, whereas the sketched affinities can only understate, so the evaluator errs toward staying; this is intended, since a move should be backed by evidence. Conversely, a net spanning many blocks consists mostly of single-pin blocks whose updates of weight $1/(|e| - 1)$ erode the top slots through the decrement rule. Updates and move candidates are therefore restricted to a candidate set $\mathcal{C}$: the two occupied blocks with the most pins of e , extended by the blocks tied with the second one up to eight blocks in total, and for nets above 128 pins the plurality block alone. When more than eight blocks are occupied, blocks in which e has no pin other than possibly v itself, that is $c_b - [\pi(v) = b] \leq 1$, receive no sketch update. Phase 2 is unaffected by $\mathcal{C}$, and so is the first candidate source of Phase 4, the blocks tracked by the sketch.

Algorithm 3 UPDATE(v, b, w): affinity update for one vertex

```
1: if  $b$  occupies a slot of  $v$  then
2:    $\text{score}(v, b) \leftarrow \text{score}(v, b) + w$  ▷ hit
3: else if  $v$  has a free slot then
4:   store  $(b, w)$  in that slot ▷ insert
5: else
6:   decrement every score of  $v$  by  $w$ ; drop the slots that reach 0 or below ▷ evict
```

Storing $a(v, \cdot)$ exactly means k counters per vertex and thus $\Theta(nk)$ memory, which for large k exceeds the size of the hypergraph and defeats the purpose of streaming. It is also more than we need. The move in Phase 4 takes an $\arg \max$, so a block toward which v is only weakly pulled can never be the answer; what we must not lose are the few blocks that carry most of v 's affinity mass. This is the *heavy-hitter* problem on a weighted stream, for which Misra and Gries [32] gave a deterministic summary of fixed size. Each vertex keeps t slots holding (block, score) pairs, and an update (b, w) is applied by the rules of Algorithm 3: increment the score if b is tracked, occupy a free slot if one exists, and otherwise decrement all scores by w , dropping the slots that reach zero or below, and discard b . We use $t = 5$ throughout.

We stress what this does and does not guarantee. For unit update weights ($w \equiv 1$; note that our updates (3) are fractional even under unit net weights) the classical analysis gives that no score exceeds the true affinity and none understates it by more than $W_v/(t+1)$, where W_v is the total weight v has received, so that every block above a $1/(t+1)$ share is retained. Carried over to weighted streams verbatim, the rule above does *not* inherit that bound: a decrement event is supposed to remove $(t+1)w$ of mass, but removes less whenever a tracked score is smaller than w ; in the worst case an event certifies only the discarded weight itself, so the understatement guarantee weakens from $W_v/(t+1)$ to W_v , that is, by the full factor $t+1$. The variant that decrements by the smallest tracked score and retains the residual restores it [7]. We use the cheaper decrement-by- w rule, which costs one subtraction per slot and no bookkeeping, and therefore treat the sketch as a heuristic estimator whose accuracy we establish experimentally rather than as one carrying a worst-case guarantee. The residual-preserving rule was rejected on cost, not on evidence: it takes the minimum over the tracked scores and carries a residual at every decrement event, a second scan of the slots and a data-dependent branch in the hottest loop of the algorithm, where decrement-by- w stays a branch-free subtraction per slot. None of the claims in Section 4 depends on the bound.

Phase 4: move evaluation. We would like to place v in the block that keeps the largest number of its nets internal, but evaluating that exactly requires $\lambda(e')$ for every net e' incident to v , and those nets are gone. We therefore score a candidate block by what is still available: the accumulated affinity, the exact evidence of the net currently in hand, the credit of v 's internal nets, and the price of growing the block,

$$s(v, b) = \underbrace{a(v, b)}_{\text{sketched history}} + \underbrace{(c_b - [\pi(v)=b]) \cdot w_e}_{\text{exact, current net}} + \underbrace{[\pi(v)=b] \cdot \text{credit}(v)}_{\text{home credit}} - \underbrace{\rho_b}_{\text{balance}}, \quad (4)$$

where $w_e = 1/(|e| - 1)$ is the normalization of Equation (3) (well-defined, since nets of size at most one are dropped on input, Section 2), $[\pi(v)=b]$ is the Iverson bracket, one if v already sits in b and zero otherwise, so that v 's own contribution to c_b (Equation (1)) is discounted (the same bracket appears in Algorithm 2 and Equation (6)), $a(v, b)$ is read from the sketch and taken to be zero

for a block it does not track, $\text{credit}(v)$ is the home credit of Phase 3, standing in for the internal nets whose updates the single-block shortcut keeps out of the sketch; since an internal net of v lies entirely in $\pi(v)$, this evidence argues only for staying, and the bracket confines it to the home score, and ρ_b is computed from the penalty table for candidates outside the cached set. The first two attraction terms are complementary: the sketch is an approximate summary over all nets of v seen so far, whereas the second term is exact but concerns one net. We move v to

$$b^* = \arg \max_{b \neq \pi(v), |V_b| < L_{\max}} s(v, b) \quad (5)$$

whenever $s(v, b^*) > s(v, \pi(v))$, and update both block sizes; b ranges over the blocks tracked by v 's sketch, extended by the candidate set $\mathcal{C}$ of the current hyperedge when $\text{seen}(v) > 5$ (Algorithm 2, line 19). Restricting the $\arg \max$ to blocks below $L_{\max}$ is what keeps the balance constraint satisfied: the penalty ρ_b discourages filling a block, but only the explicit bound enforces it. Since Phase 3 has just loaded and updated the sketch of v , the evaluation reuses data that is already in cache, and fusing the two phases into a single loop over the pins avoids a second random access per pin.

Three restrictions keep such moves useful. First, the sketch is an approximation and may propose a move that is demonstrably harmful for the net in hand. Measured against the counts we maintain, moving v from $\pi(v)$ to b' increases $\lambda(e)$ precisely when $c_{\pi(v)} > 1$, so that v is not the last pin of e in its block, and simultaneously $c_{b'} = 0$, so that entering b' makes e span one more block. We reject such moves, which prevents accumulated history from overriding local, certain harm. Second, a vertex is eligible only once $\text{seen}(v) \geq 2$ and its sketch tracks at least two blocks, since a vertex seen in a single net of the current pass has no basis for revising its own placement. The second condition is a property of the sketch rather than of the vertex, and it is conservative: a high-degree vertex whose sketch has been reduced to one tracked block is skipped even though the current net would supply candidates. Third, with only t slots the sketch cannot track all relevant blocks of a high-degree vertex, so for $\text{seen}(v) > 5$ we additionally consider the candidate blocks $\mathcal{C}$ of the current net that the sketch does not track. The thresholds 2 and 5 are design parameters, fixed in preliminary experiments on the tuning instances and not varied further; Section 4.1 examines the choice of t .

Most evaluations end before any candidate is scored. The score of any block other than $\pi(v)$ is at most the largest sketch score of such a block, plus $\max_b c_b \cdot w_e$ for the edge-local term, minus a lower bound on the smallest penalty of any block, namely the penalty of the least-loaded block as cached at the start of the net or as it currently stands, whichever is smaller; this is the bound $\bar{s}(v)$ of Algorithm 2. If this bound does not exceed the home score, which includes the home credit, no move is possible and the candidates are not scored. The bound is valid, so the test changes no decision. Note that the counts c_b are *not* updated when a move is executed, so pins processed later in the same hyperedge are scored against the block distribution as it stood after Phase 2; keeping c exact within the loop would cost a second pass and, in preliminary experiments, did not pay for itself.

3.2 Prioritized Buffering with an Incremental Priority Queue

The first hyperedge of a stream arrives when no vertex has been assigned, so Equation (2) degenerates: all co-pin counts are zero, the attraction term carries no information, and the placement is decided by the balance penalty alone, that is, arbitrarily. This would not matter if the damage stayed local, but it does not. Every later hyperedge that shares a pin with an early one inherits those arbitrary placements through its co-pin counts, so the noise of the first decisions propagates through the stream.

We therefore do not decide immediately when there is nothing to decide from. A hyperedge whose fraction of already-assigned pins falls below $\theta = 0.7$ is placed into a buffer of capacity $B = 2^{14}$ hyperedges instead of being processed; when the buffer is full, or the pass ends, it is flushed in its entirety, and hyperedges are never re-buffered. At the start of pass 0 nothing is assigned, so every early net falls below θ ; the first assignments of a run are therefore made inside the first flush (on instances with fewer than B nets, at the end of the pass), seeded by a net whose pins are all new and are placed by the balance penalty alone, and the flush order below grows the partition outward from that seed. The order of the flush is what matters. Processing the buffered hyperedge with the largest assigned fraction first is the obvious choice, but a single sort is not enough, because the ordering is stale the moment it is computed: processing one hyperedge assigns its new pins, which raises the assigned fraction of every buffered hyperedge sharing those pins.

We therefore maintain the priority order incrementally. Two structures over the buffered hyperedges make this possible: a bucket queue, which always yields the currently best-informed hyperedge, and a vertex-to-edge index, which locates the hyperedges whose priority a placement has just changed. The queue keys every hyperedge by its assigned fraction in steps of 1%, that is 101 buckets, most recent first within a bucket; the index lists, for every still-unassigned pin, the buffered hyperedges containing it, assigned pins being left out since only the later assignment of an unassigned pin can raise a fraction, and is stored as a flat linked list, or in compressed sparse row form with software prefetching once it covers more than a million (2^{20}) pins. A flush then repeats two steps until the queue is empty: it extracts a hyperedge from the highest non-empty bucket and processes it with the four phases of Algorithm 2, which assigns its unassigned pins; and for each pin this has just assigned, it traverses that pin’s index list and increments the counter of every other buffered hyperedge containing the pin, re-inserting the hyperedge into the queue whenever its bucket changes. Two details keep the bookkeeping linear: the list of a pin is emptied the first time it is traversed, so each incidence is followed at most once over the whole flush, and a re-inserted hyperedge leaves its old queue entry behind, to be filtered by a processed flag on extraction. Building the index costs $O(P_B)$ for a buffer holding P_B pins, every hyperedge is processed exactly once, and every update is $O(1)$, so the queue and index maintenance of a flush costs $O(P_B + B)$ time, on top of the per-net processing of the four phases, which the pass bound of Section 3.6 covers; the index is what a single sort would not need, and the justification of the incremental order is the quality of the resulting decisions, not its asymptotic cost. The effect compounds: each processed hyperedge raises the priority of its neighbors, so a hyperedge that entered the buffer blind is typically better informed by the time it is dequeued. The most-recent-first order within a bucket matters as well: it grows one region of the hypergraph at a time, whereas a first-in-first-out order within a bucket, like a level-synchronous flush that sweeps the buffer with descending thresholds, mixes regions and blurs this effect. A higher threshold buffers more nets at a higher cost of the flush; $\theta = 0.7$ is the compromise we use, and Section 4.1 examines the trade-off.

The buffer stores pin lists and the vertex-to-edge index, and therefore occupies $\Theta(P_B + n)$ words, n of them the heads of the index, in addition to the per-vertex state, where P_B is the number of buffered pins. Its capacity is a constant number of hyperedges, $B = 2^{14}$, so $P_B \leq \min(P, B \cdot \Delta_E)$, where Δ_E is the largest net size: the buffer depends on the input through n and the sizes of the nets it holds, never through m . Buffered nets above 1024 pins are kept in a separate list that is processed, in stream order, after the queue has been emptied (they count toward the capacity B), and left out of the vertex-to-edge index, since their pin counts carry little placement signal and would merge far-apart vertices before the small nets around them arrive. Section 4.1 reports how the peak memory of the whole process depends on the capacity. The capacity is a trade-off examined in Section 4.1: a smaller buffer loses quality, a larger one buys up to 8% at proportionally

more time and memory. Section 3.6 accounts for it next to the persistent state, and it is included in the memory measurements of Section 4. Buffering is active on the first pass (pass 0) only: from the second pass onward every pin is assigned, so no hyperedge falls below θ .

3.3 The Final Sketch Sweep

During the pass a vertex is re-evaluated only when one of its nets arrives, so its last decision is taken against a sketch that is still incomplete, and a vertex whose nets all arrived early never sees the evidence that arrived later. Once the pass has ended the sketches are as complete as they will get, and a sweep re-evaluates every assigned vertex whose sketch tracks at least two blocks (the $\text{seen}(v)$ gate of Phase 4 does not apply) with the move rule of Phase 4, reduced to the sketch, the home credit and the balance penalties since no net is in hand, and moves it to its best tracked block if $|V_b| < L_{\max}$; this is where the ε slack reserved by the placement cap is spent. The sweep runs in rounds until fewer than 0.5% of the vertices move or 20 rounds are done, needs no access to the stream, and costs $O(n \cdot t)$ per round; the cap is needed because the sweep decreases no potential and can therefore cycle, a few vertices moving back and forth as the block sizes, and with them the penalties, alternate. Later rounds are cheap because a vertex’s sketch changes only when the vertex itself moves: the size penalties of any two blocks differ by at most $\alpha\gamma(L_{\max}^{\gamma-1} - L_{\min}^{\gamma-1})$ as long as no block is smaller than $L_{\min}$, so a vertex whose home score exceeds its best other slot by more than this gap cannot move in a later round and is flagged and skipped. The flags assume the lower bound $L_{\min} = \min_b |V_b| - \max(2, \lceil 0.02 \min_b |V_b| \rceil)$ on the block sizes, set at the start of each round; they are cleared should any block shrink below $L_{\min}$, so the decisions are exactly those of a sweep without flags. The sweep repairs the decisions that the pass had to take on incomplete sketches; what it cannot repair are moves to blocks that the sketch does not track.

3.4 Restreaming

Everything so far sees each hyperedge once, which is what the one-pass model demands. A hypergraph that resides on disk, however, can simply be read again, and the algorithm then gets to revisit decisions it had to make blind: the pins placed early in the first pass were placed when almost nothing was known, and by the end of that pass the sketches hold evidence from the whole instance. We therefore keep the assignment and stream the hyperedges again, so that Phase 4 can revise placements against a complete, if approximate, picture, and we repeat this for a fixed number of passes. The sweep of Section 3.3 already gives every vertex one such revision without re-reading the input. The evaluation of Section 4 therefore uses a single pass followed by the sweep as the default and reports restreaming with three and ten passes alongside it. Restreaming remains the option for inputs that can be read again, and we describe it here because it raises the question of which state a pass carries over. Following **ReFennel** [34], we raise the balance penalty from pass to pass, using $\alpha_r = (1 + 0.3r)\alpha$ in pass r , where the first pass is $r = 0$ and therefore uses the α of Equation (2) unchanged; this tightens balance as the partition settles; each pass, including the first, ends with the sweep of Section 3.3, which uses the same α_r . Two questions arise that a single pass never poses, namely which state to carry across passes and which hyperedges to re-read at all.

Sketch accumulation. The decisive design choice is that the sketches are *not* reset between passes. Resetting them is tempting, since the partition has changed and the accumulated affinities were measured against an older one, but it is harmful: a vertex then enters the new pass with an empty sketch, its first few moves are decided on almost no evidence, and those noisy moves change

the co-pin counts seen by later hyperedges, which cascades. Accumulating instead means every pass adds evidence to the same estimates. We emphasize that nothing here forces the objective to decrease: moves are accepted on approximate scores rather than on exact objective deltas, so a pass may in principle make the partition worse (and does so on 27% of the instance- k pairs of the benchmark of Section 4.2 after three passes and on 33% after ten, against 71% and 66% on which it is better), and unlike **ReFenne1** we have no convergence proof. Section 4.1 compares the two choices on the component-study set; accumulation is the default. Only $\text{seen}(v)$ is reset each pass, since it gates move eligibility within a pass rather than accumulating evidence; the home credit is kept, as it is folded into the sketch and cleared only when the vertex moves. A consequence of the reset is that a vertex must be seen twice in the new pass before it may move again within the pass, even though its sketch is fully populated.

Skipping internal hyperedges. A restreaming pass as described re-reads all m hyperedges, including those that have become internal. The single-block skip already prevents such a hyperedge from reaching Phases 2–4, so it produces no affinity updates and no moves; what it still costs is Phase 1, which reads the record of every pin *and writes* to it, since classification increments $\text{seen}(v)$ and the shortcut adds the home credit. The writes are what the pre-test avoids: each dirties a cache line that must later be written back, whereas a read-only scan leaves the line clean.

On every restreaming pass, that is for $r \geq 1$, we therefore test whether all pins of a hyperedge share a block before entering Phase 1, and skip it if they do. The test reads the same records but writes none, and it aborts at the first pin that disagrees, so it is strictly cheaper than Phase 1 for internal hyperedges and adds a partial scan for the rest. We stress that this is a running-time optimization and not, as one might expect, a way to keep uninformative updates out of the sketches: by the single-block skip, internal hyperedges never reach the sketches in the first place. It is nevertheless not quality-neutral, because skipping a hyperedge also suppresses the $\text{seen}(v)$ increments and the home credit it would have contributed, which makes its pins less likely to become move-eligible in that pass; and since the credit is folded only on a move while cut nets keep adding sketch mass in every pass, the weight of the home block relative to the tracked alternatives declines over the passes for vertices whose internal nets are skipped. We keep the skip as the default measured throughout Section 4, and note that inputs read from slow storage, where the pass cost is dominated by I/O anyway, are better served without it.

3.5 Optimizing the Cut-Net Metric

The connectivity metric charges a hyperedge for every block it spans, whereas the cut-net metric charges it once, no matter how badly it is split. Under the cut-net metric, reducing a hyperedge from four blocks to two gains nothing, and only making it internal counts. The move score (4) needs no change to reflect this; it suffices to change what the sketch measures. In cut-net mode the sketch update replaces the connectivity weight (3) by the squared ratio of the co-pins of v in b to the pins of e that were assigned when e is processed,

$$w_{\text{cut}}(v, b) = \left(\frac{c_b - [\pi(v)=b]}{A_e - 1} \right)^2, \quad (6)$$

where A_e counts the pins of e that were already assigned when e is processed (for a buffered net, when it is dequeued), the subtraction of one discounts v itself (one pin too many if v was new), and c_b is the co-pin count after Phase 2, so that pins placed by e itself count as evidence.

The intent is easiest to see for a net without new pins. There the ratio is the fraction of the other pins of e that sit in b , and squaring concentrates the signal on the nets that a single move can make internal, the only event the cut-net metric rewards: a block that holds all other pins of e receives the full unit, one that holds half of them a quarter, whereas the connectivity weight (3) would credit both in proportion. The signal is nevertheless graded rather than binary, and deliberately so: a signal that is nonzero only for nets one move away from internal would leave the sketches of most vertices empty and the move evaluator without candidates.

Two boundary cases remain. For a net with new pins the ratio can exceed one, since the numerator counts the pins that Phase 2 has just placed into b . In stream order it stays at most 1.5, since $A_e \geq \theta|e|$, whereas for the first nets of a flush, where A_e can be 2, it reaches $|e| - 1$ and w_{cut} reaches $(|e| - 1)^2$. And no update is made when $A_e \leq 1$, which occurs only for the first nets of a flush, since a net processed in stream order has $A_e \geq \theta|e|$; in particular, a net all of whose pins are new has $A_e = 0$ and makes no update: its pins carry no evidence yet and receive their first from later nets.

3.6 Memory Layout and Resources

What remains is to account for the cost, and layout and running time are one question rather than two: the accesses in Phases 1, 3 and 4 are random and the arithmetic per pin is a handful of comparisons, so the time is essentially the number of cache lines a hyperedge touches, and the same state is also almost all of the space.

Compact per-vertex layout. We therefore interleave the per-vertex state into two records, a hot one and a cold one, so that a single fetch serves the fields that every pin touches, and a second one the sketch, which only a cut net needs, instead of one miss per array. Types are chosen per instance: the block identifier type `BlockId_t` is selected at compile time as `uint8_t` for $k < 256$, `uint16_t` for $k < 65536$ and `int32_t` otherwise, and the same type stores the partition, with the largest representable value as the sentinel for unassigned (block indices are stored 0-based); the counter $\text{seen}(v)$ is a saturating `uint16_t`, which is harmless since it is only compared against the thresholds 2 and 5. Writing β for the width of `BlockId_t` in bytes, the hot record holds the partition (β B), $\text{seen}(v)$ (2 B) and the home credit (4 B), padded to 8 bytes for $\beta \leq 2$ and packed contiguously at natural alignment, eight records per 64-byte cache line rather than one padded to it, since the passes that touch them read sequentially, so that the classification pass, the internal-net shortcut and the buffering decision read eight vertices per 64-byte line and never pull a sketch into the cache. The cold record holds the sketch: t block identifiers ($t\beta$ B) and t scores ($4t$ B), the slot count and the effective slot count (1 B each), an eviction counter (2 B) and mirrors of $\text{seen}(v)$ and the partition (the effective slot count, the eviction counter and the $\text{seen}(v)$ mirror serve only diagnostics and adaptive variants that are not enabled, 5 of the 49 bytes per vertex; a re-implementation can omit them without changing any output, leaving 44 bytes, whereas the partition mirror is load-bearing for the cache locality of the sweep and the flush (Table 1 counts all 49, since every measurement of Section 4 runs the binary with the diagnostic fields in place)); with $t = 5$ this is 32 bytes at $\beta = 1$ and 40 bytes at $\beta = 2$, and the slot count is fixed at compile time so that the block identifiers are compared with one SIMD instruction. The cold record mirrors the partition, kept in lockstep on every write, so that the sketch-side code of the sweep and the flush reads a vertex’s block from the record already in cache; the $\text{seen}(v)$ mirror is refreshed once per pass and serves only diagnostics. The slot search over the block identifiers is one explicit 128-bit SSE compare across all t slots for $\beta \leq 2$ (a scalar loop otherwise); the score loops are fixed-size

Component	Asymptotic	Bytes ($t=5, \beta=2$)
<i>Hot per-vertex record (8 bytes):</i>		
Partition (<code>BlockId_t</code>)	$O(n)$	$\beta n = 2n$
Seen counter <code>seen(v)</code> (<code>uint16_t</code>)	$O(n)$	$2n$
Home credit (<code>float</code>)	$O(n)$	$4n$
<i>Cold per-vertex record (40 bytes):</i>		
MG block IDs (<code>BlockId_t</code>)	$O(n \cdot t)$	$\beta t n = 10n$
MG scores (<code>float</code>)	$O(n \cdot t)$	$4t n = 20n$
Slot counts, eviction counter	$O(n)$	$4n$
Seen-counter and partition mirrors	$O(n)$	$4n$
Struct padding	–	$2n$
<i>Separate per-vertex array:</i>		
Sweep flags (<code>uint8_t</code>)	$O(n)$	n
<i>Per-vertex subtotal</i>	$O(n \cdot t)$	$49n$
Block sizes, size order and positions	$O(k)$	$16k$
Penalty lookup table (8 B), size boundaries (4 B)	$O(n/k)$	$\approx 12L_{\max}$
Scratch (count, penalty cache, candidates)	$O(k)$	$\approx 24k$
Total	$O(n \cdot t + k)$	$\approx 49n + 40k + 12L_{\max}$
<i>Transient: first-pass buffer (pins and index)</i>	$O(\min(P, B \cdot \Delta_E) + n + B)$	$\approx 8P_B + 4n$

Table 1: Persistent state of HedgeStream, with the transient first-pass buffer of Section 3.2 listed separately. Here t is the number of Misra–Gries (MG) slots per vertex, $P_B \leq \min(P, B \cdot \Delta_E)$ the buffered pin count of Section 3.2, and β the width of `BlockId_t` in bytes; the column instantiates $256 \leq k < 65536$, where $\beta = 2$; at $\beta = 1$ (the type serves $k \leq 255$ with the sentinel; in our grid, $k \leq 128$) the total is 41 bytes per vertex.

and scalar, left to the compiler at the optimization flags of Section 4. For $k \leq 2$ the sketch never evicts, so it holds the affinities as defined through $\mathcal{C}$ exactly.

Time. Consider a hyperedge with $p = |e|$ pins, $|new|$ of them unassigned (Algorithm 2), and $q \leq \min(p, k)$ occupied blocks. Classification and counting cost $O(p)$, and resetting the sparse counter afterwards costs $O(q)$. Assignment evaluates q candidates per new pin plus the least-loaded block, which is $O(|new|q)$. Phase 3 performs one update per (pin, candidate block) pair and each update scans t slots; since the candidate restriction of Section 3.1 admits a constant number of candidate blocks, the phase costs $O(pt)$, and the fused Phase 4 adds $O(pt)$ as well. Now sum over the stream. The terms linear in p add up to $O(tP)$, where $P = \sum_e |e|$ is the pin count, and the $O(q)$ resets are absorbed by them, since $q \leq p$. The assignment term is best summed over the vertices rather than the nets: every vertex is unassigned for exactly one processed net, so $\sum_e |new| \cdot q$ adds up, over the n placements of the pass, the number of occupied blocks q at the moment of that placement. Writing $\bar{q}$ for the average of these n values, one pass costs

$$T_{\text{pass}} = O(tP + n\bar{q}). \quad (7)$$

R passes cost R times as much, plus $O(P_B + B)$ per buffer flush and $O(n \cdot t)$ per round of each sweep. The bound depends on k only through $\bar{q}$; what varies with k in practice is the share of nets that take the internal-net shortcut of Phase 1, which Section 4.3 quantifies.

Constant	Value	Fixed by
Sketch slots t	5	component study (Section 4.1)
Buffering threshold θ	0.7	component study
Buffer capacity B	2^{14} nets	component study
Passes	1 (3, 10 as variants)	component study
Load-penalty scale in α	0.7	preliminary experiments, tuning set
Fennel exponent γ	$3/2$	inherited from Fennel
Move eligibility	$\text{seen}(v) \geq 2, \geq 2$ tracked blocks	hand-set
Candidate extension	for $\text{seen}(v) > 5$	hand-set
Candidate-set cap	8 blocks	hand-set
Large-net rules	128 pins (plurality update), 1024 (side list)	hand-set (effect: Table 4)
Sweep stopping rule	$< 0.5\%$ moves or 20 rounds	hand-set; rounds: Table 6
Sweep size guard	$L_{\min} = \min_b V_b - \max(2, \lceil 0.02 \min_b V_b \rceil)$	hand-set

Table 2: Every constant of the implementation with its value and its origin: the first four and the sweep-round cap were fixed on the tuning instances and varied in the component study of Section 4.1; the remaining ones were set by hand or in preliminary experiments on the same instances and are not varied.

Space. The persistent state is $O(n \cdot t + k)$, dominated by the per-vertex records (Table 1) and independent of both m and P ; only the first pass adds the transient buffer of Section 3.2, which holds at most $B = 2^{14}$ hyperedges, that is $P_B \leq \min(P, B \cdot \Delta_E)$ pins at about eight bytes each (four for the pin list and four for the vertex-to-edge index), $4n$ bytes for the heads of the index plus $O(B)$ words for the bucket queue; Section 4.1 reports how the peak memory depends on the capacity. Of the persistent state the partition attains a counting lower bound: any algorithm that maintains and revises an assignment of n vertices to k blocks must keep $n \log_2 k$ bits, which our **BlockId_t** encoding attains up to byte alignment and which saves a factor of two ($\beta = 2$) or four ($\beta = 1$) over the natural **int32_t**; the bound is one for this class, not for the problem, since an algorithm that emits its assignments irrevocably, such as **Hashing**, keeps no partition at all; for the sketch we claim no optimality, as t is a design parameter. This is the structural difference to the vertex-stream model, where **FREIGHT** keeps, per net, the last block assigned from it together with a cut-status flag, so that $\Theta(m)$ state persists for the whole run and its memory follows the number of nets while ours follows the number of vertices. Whenever m exceeds n several times over, the break-even at the constants of Table 1 against **FREIGHT**’s measured 8 to 17 bytes per element of $V \cup E$ (its peak resident set size, RSS, divided by $n + m$; the upper end reflects fixed overheads on the smaller instances) lying between $2n$ and $5n$ (68 of the 310 benchmark instances lie beyond $2n$ and 28 beyond $5n$), as on many of the SAT instances (Section 1), **HedgeStream**’s persistent state is the smaller of the two; on the first pass its peak additionally carries the buffer.

4 Experimental Evaluation

Setup. We implemented **HedgeStream** in C++ and compiled it with g++ 13.3 at full optimization (`-O3 -march=native`), with exceptions, run-time type information and non-finite arithmetic disabled. All experiments are performed on a single core of a machine with a 128-core AMD EPYC 9754 processor clocked at up to 3.1 GHz, 755 GiB of main memory, 32 KiB of L1 data cache and 1 MiB of L2 cache per core, and 256 MiB of L3 cache, running Ubuntu 24.04. All baselines were built with the same compiler at the same optimization level, with the flags of their own build systems. Each algorithm uses a single core of the machine. The median coefficient of variation

of the measured times over the ten repetitions of every run of the main evaluation (see below) is 0.6% for HedgeStream, 1.1% for **FREIGHT** and 1.4% for **Hashing**. Sections 4.1 to 4.5 stream every hypergraph directly from internal memory in order to obtain clear running time comparisons: HedgeStream’s kernel consumes one net at a time, but the driver used there reads the file into memory before the pass, and the baselines hold their input in memory as well. Section 4.6 measures the true streaming setting: HedgeStream’s streaming driver reads one net at a time from the file, and **FREIGHT**, which can equally stream from disk, does so there too.

Baselines. We compare against the same set of algorithms that was used to evaluate **FREIGHT**, extended by **FREIGHT** itself and by **SIGHP** [23]. Our main baseline is **FREIGHT** [17], the state of the art for streaming hypergraph partitioning with respect to connectivity and cut-net, which we run in both its connectivity and its cut-net configuration. It is the natural point of comparison because it optimizes the same two objectives under a balance constraint and because it defines the vertex-stream model whose costs motivate this work (Section 1). We further compare against **Hashing**, which assigns the identifiers of incoming vertices to $\{1, \dots, k\}$ by a hash function and which is relevant because it is the simplest and fastest streaming algorithm and therefore gives a lower bound on partitioning time. We include **Min-Max** [2] and all of its improved and buffered versions proposed by Taşyaran et al. [44], namely **Min-Max-N2P**, **Min-Max-L ℓ** , **REF**, **REF_RLX** and **REF_RLX_SV**, since **Min-Max** is the other established streaming baseline for these objectives, some of its variants are orders of magnitude faster than it, and others produce better partitions than it (see Section 2.2). We ran the remaining two variants, **Min-Max-BF** and **Min-Max-MH**, as well, but they are the two worst algorithms of the **FREIGHT** study [17] on both objectives and were so in our runs, so we do not present them here.

Finally we compare against the in-memory partitioners **HYPE** [29], **SIGHP** [23] and **PaToH v3.3** [11], which are not suitable for the streaming setting but serve as reference points, **HYPE** because it is among the fastest in-memory algorithms, **SIGHP** because it is the most recent single-threaded partitioner built for very large hypergraphs and reports large gains over **FREIGHT** on its own benchmark, and **PaToH** because it is fast and computes partitions of very good quality; **KaHyPar** [40, 41] is the leading tool with respect to quality: its flow-based configuration computes partitions with a connectivity that is on average 8.7% better than that of **PaToH** in its quality preset and 14.4% better than in its default preset, and it is better on more than 90% of the instances of its benchmark; it is, however, one to two orders of magnitude slower than **PaToH**, which puts running it on our 310 instances at six values of k out of the scope of this study, so we do not include it. Its published per-instance results¹ nevertheless quantify the gap on our benchmark: on the 1 164 instance- k pairs that data shares with our grid (294 of our 310 instances at $k \in \{2, 8, 32, 128\}$, the same $\varepsilon = 0.03$, ten seeds; the 16 remaining instances, among them the largest of the benchmark, are absent from their data because some of the tools evaluated in [40] ran out of memory or time on them), **kKaHyPar** computes connectivities better than HedgeStream’s by a factor of 4.0 in geometric mean, from 6.6 at $k = 2$ to 2.5 at $k = 128$, and better than our **PaToH** runs by 1.16, consistent with the factors above; it takes 38 times **PaToH**’s time on their machine, 28 s per pair in geometric mean under an eight-hour limit, about 155 hours for a single seed of that shared subgrid alone.

Since **FREIGHT** and the **Min-Max** variants consume vertex-centric formats, we convert every instance beforehand with the converter shipped with the respective tool and exclude the conversion from the reported running times, which favors those algorithms. All binaries were built from the authors’ public repositories with the compiler settings described above: **FREIGHT** and **Hashing** are

¹<https://github.com/kahypar/experimental-results>, folder jea20.

the netlist-streaming tools of the **FREIGHT** repository, run with the settings of [17] and with a flag `-skip_evaluation` that we added to their sources: without it the tools evaluate the partition inside the same process after partitioning, which re-reads the input and keeps a set of touched nets per block, and the flag skips this and leaves the partition and the kernel time unchanged (the objectives come from the common evaluator described below); the flag removes only code that runs after the partition is complete, so partition and kernel time are unaffected by construction, and every reported measurement of the two tools, including peak memory, is taken with the flag, the **Min-Max** family and **PaToH** are driven through the implementation of Taşyaran et al. with a buffer of 15% of the pins for the buffered variants and **PaToH** in its default preset, and **SIGHP** is the authors’ optimized source with its default hub shielding, fed through a bipartite edge-list converter that we wrote for the hMetis format, whose conversion time is, like the vertex-stream conversions, excluded from the reported times; **HYPE** and **SIGHP** have a single configuration each, whose partitions we evaluate under both objectives. The **Min-Max** variants use $\ell = 5$, the better of the two values tested in [44]. HedgeStream runs a single pass followed by the sketch sweep of Section 3.3, with the parameters of Section 3 ($t = 5$, $\theta = 0.7$, $B = 2^{14}$), in its connectivity configuration (HedgeStream-con) and in the cut-net configuration of Section 3.5 (HedgeStream-cut); we also run its restreaming configurations of Section 3.4 and, as a diagnostic, its buffer-free configuration ($B = 0$) under both objectives (Section 4.2).

Instances. We use the same benchmark as [40], which consists of 310 hypergraphs from three sources: 18 hypergraphs from the ISPD98 Circuit Benchmark Suite [3], 192 hypergraphs based on the University of Florida Sparse Matrix Collection [14], forming the sparse-matrix (SPM) family, and 100 instances from the international SAT Competition 2014 [6], the SAT14 family. The SAT instances are converted into hypergraphs by mapping each boolean variable and its complement to a vertex and each clause to a net. From the Sparse Matrix Collection, one matrix was selected per application area with between 10 000 and 10 000 000 columns, and the matrices are converted using the row-net model, in which each row is a net and each column a vertex. The benchmark files come with parallel and empty hyperedges removed and unit weights on all vertices and nets; single-pin nets remain in the files and are counted in the statistics below and in Table 3, whereas our implementation drops them on input (Section 2). The instances span three orders of magnitude in size: n ranges from 10 000 to 9 845 725 vertices, m from 163 to 13 378 625 nets, and the pin count from 6 323 to 113 891 327, 1.4 billion pins in total; on 128 of the 310 instances there are more nets than vertices, which is the regime in which the vertex-stream model keeps more per-net state than the partition itself. Section 4.6 adds four giant instances an order of magnitude beyond the benchmark: the web hypergraphs **webbase-2001**, **it-2004** and **sk-2005** from the same collection [14] in the row-net model, with 118, 41 and 51 M vertices and 1.02, 1.15 and 1.95 billion pins, and the SAT formula **sat_nurikabe_p18** from the Global Benchmark Database in the primal model of the SAT14 family, with 29 M variables, 352 M clauses and 0.81 billion pins; they are streamed from disk only (Table 12). Prior to each experiment we converted all hypergraphs into the streaming format required by the respective algorithm, and we stream every instance in the natural given order of its nets, that is, in the order of the file, without any permutation. Unless mentioned otherwise we use a number of blocks $k \in \{2, 8, 32, 128, 512, 2048\}$, which covers the range from few to many blocks on a geometric scale and includes two of the large values at which **FREIGHT** was evaluated. We allow a fixed imbalance of $\varepsilon = 3\%$ for all algorithms and experiments, since this is a frequently used value in the partitioning literature.

Family	Instance	n	m	Pins	Avg. $ e $	Max. $ e $	m/n
ISPD98	ibm01	12 752	14 111	50 566	3.6	42	1.11
	ibm02	19 601	19 584	81 199	4.1	134	1.00
	ibm07	45 926	48 117	175 639	3.7	25	1.05
	ibm16	183 484	190 048	778 823	4.1	40	1.04
	ibm17	185 495	189 581	860 036	4.5	36	1.02
SAT14	AProVE07	15 004	28 770	76 290	2.7	847	1.92
	6s184	66 730	97 516	227 536	2.3	3	1.46
	atco_enc1	118 700	561 784	2 167 217	3.9	11	4.73
	E02F22	27 148	1 301 188	11 462 079	8.8	78	47.93
	9vliw_bug10	1 042 364	13 378 625	39 204 931	2.9	6 689	12.83
SPM	Pd_rhs	12 406	4 371	6 323	1.4	14	0.35
	msc10848	10 848	10 848	1 229 778	113.4	723	1.00
	gupta3	16 783	16 783	9 323 427	555.5	14 672	1.00
	Dubcova2	65 025	65 025	1 030 225	15.8	25	1.00
	Emilia_923	923 136	923 136	41 005 206	44.4	57	1.00

Table 3: The fifteen representative instances used for the component study and for tuning: number of vertices n , nets m , pins, average and maximum net size, and the ratio of nets to vertices. The statistics are those of the files and include single-pin nets, which the implementation drops on input; Pd_rhs consists mostly of them, so most of its vertices are isolated and are placed at the end of the run into the lightest block.

Methodology. Depending on the focus of the experiment we measure connectivity, cut-net, running time, or peak resident memory. Running times are the kernel times reported by the tools (Section 4.3 states the convention) and are normalized per pin, that is, divided by the number of pins of the instance as the file stores it, including single-pin nets (Table 3), wherever algorithms are compared across instances. We perform ten repetitions per algorithm and instance. We use the geometric mean throughout, except where medians are explicitly stated (Section 4.3), both to aggregate the repetitions of a single instance and to average over multiple instances, so that every instance has the same influence on the final score. Given a result of an algorithm $\mathcal{A}$, we express its value $\sigma_{\mathcal{A}}$, which may be an objective or a running time, as an *improvement* over an algorithm $\mathcal{B}$, computed as $(\frac{\sigma_{\mathcal{B}}}{\sigma_{\mathcal{A}}} - 1) \cdot 100\%$. Altogether the evaluation comprises 390 600 runs, the 21 presented algorithm configurations on 1 860 instance- k pairs with ten repetitions each, and every run completed within its limits of 24 hours and 40 GiB.

Objectives are computed by one common evaluator from the written partition; in 70 runs of each one-pass HedgeStream and FREIGHT configuration (80 for the restreaming configurations), 140 runs of PaToH and 19 runs of the Min-Max family the objective is zero (a pair contributes ten repetitions, so 70 runs are seven pairs), most of them at $k = 2$ and the rest at $k \leq 128$, and such values are replaced by one before the geometric mean is taken, which keeps the pair in the mean, so that every algorithm is aggregated over the same set; eight pairs are affected for HedgeStream and FREIGHT together (seven each, six of them shared), and excluding them changes the connectivity ratio between the two from 1.510 to 1.504 and the cut-net ratio from 1.147 to 1.142.

The balance constraint is checked against $\lceil (1 + \varepsilon)n/k \rceil$ (the $L_{\max}$ of Section 2 at unit weights) with a tolerance of 10^{-5} ; HedgeStream and FREIGHT respect it in every run, SIGHP and PaToH in all but 0.1% of their pairs, whereas Hashing violates it in 22% of its pairs, all at $k \geq 512$, and by up to a factor of three, and HYPE and the Min-Max variants in 3 to 6% of their pairs. A pair in which the majority of an algorithm’s repetitions is infeasible counts as unsolved in the performance profiles and is excluded from the head-to-head counts; the geometric means over all pairs include

it, which changes them by at most 2.7 percentage points of improvement over **Hashing** for the **Min-Max** family and 6.4 for **HYPE**. Since **Hashing** itself is infeasible on 22% of all pairs, every one of them at $k \geq 512$ (about two thirds of the large- k pairs), the improvements over **Hashing** at large k are measured against an infeasible baseline there; the direct ratios between feasible algorithms, including every HedgeStream–**FREIGHT** ratio, do not involve **Hashing**.

The 15 instances on which HedgeStream’s parameters were tuned (Table 3) are part of the benchmark; we therefore report the headline ratios also on the 295 other instances. Where we compare two algorithms directly we report the geometric mean ratio over all pairs together with a 95% confidence interval from a bootstrap over the 310 instances (1 000 replicates) and a paired Wilcoxon signed-rank test on the per-instance log-ratios. We also use *performance profiles* to present results. They relate the quality (running time) of a group of algorithms to the best (fastest) one on a per-pair basis rather than grouped by k . The x -axis shows a factor τ , while the y -axis shows the fraction of instance- k pairs for which the algorithm in question is within a factor τ of the best (fastest) algorithm on that pair; the value at $\tau = 1$ is thus the fraction of pairs on which an algorithm is the best one. In addition we use line charts to visualize an objective in relation to k , and box plots to show the distribution of running times, displaying minimum, maximum, median and the first and third quartiles while disregarding outliers.

4.1 Impact of the Engineering Components

Before comparing HedgeStream with the other algorithms, we quantify what each of its components contributes. All series of this section measure connectivity; the cut-net configuration of Section 3.5 inherits the parameters fixed here. The study uses the fifteen instances of Table 3, a representative set of five instances per family, each partitioned for the six values $k \in \{2, 8, 32, 128, 512, 2048\}$; we call each instance- k pair a *cell*, so that every series covers 90 cells. They span the benchmark in size, in the ratio m/n of nets to vertices, which decides how much state the vertex-stream model has to keep, and in the net-size distribution, from the two- and three-pin nets of circuits and SAT clauses to the dense rows of **gupta3** with 555 pins on average and 14 K at most. Each component is switched off in turn while all others stay at their defaults, and we report the connectivity and the running time relative to the full configuration, per k and per family; a second series adds the components

Component switched off	Connectivity relative to the full configuration							Time
	$k=2$	8	32	128	512	2048	All	rel.
Prioritized buffering (process in stream order)	1.90	1.56	1.42	1.20	1.08	1.01	1.33	0.73
Incremental flush order (sort once instead)	1.88	1.43	1.26	1.15	1.05	1.02	1.27	0.92
Home credit	1.57	1.46	1.33	1.25	1.10	1.02	1.27	0.98
Placement cap $\lceil n/k \rceil$	1.02	1.02	1.02	1.00	1.00	1.00	1.01	0.98
Candidate restriction on dense nets	1.00	0.98	1.06	1.05	1.12	1.08	1.05	1.30
Large-net rules (128, 1024 pins)	1.02	1.01	1.03	1.02	0.99	0.99	1.01	1.03
Final sketch sweep	1.04	1.27	1.16	1.10	1.17	1.10	1.14	0.87
Sketch size $t = 5$ (use 3)	1.00	1.03	1.14	1.05	1.05	1.05	1.05	0.94

Table 4: Effect of switching off one component at a time on the fifteen instances of Table 3: geometric mean connectivity relative to the full configuration per k and over all 90 cells (values above one are worse), and geometric mean running time relative to the full configuration over all 90 cells. The exact pruning bound of Section 3.1 is output-identical and therefore not part of this table.

one at a time in the order in which they entered the algorithm, a third series varies the sketch size (Section 3.1), the buffering threshold (Section 3.2) and the sweep (Section 3.3), isolating the interaction of sketch and sweep, a fourth series varies the number of restreaming passes (Section 3.4), and a fifth series varies the capacity of the first-pass buffer around the default $B = 2^{14}$.

Leave-One-Out Series. Table 4 shows the leave-one-out series. As a reference point, the full configuration reaches a geometric mean connectivity of 0.66 relative to **FREIGHT-con** on the 90 cells, ranging from 0.39 at $k = 2$ to 1.00 at $k = 2048$. *Prioritized buffering* is the largest single component: without it every net is decided in stream order, the first nets of a stream are placed blind, and connectivity rises by 33% overall, with the loss largest at small k ($1.90\times$ at $k = 2$, shrinking monotonically to $1.01\times$ at $k = 2048$, where the balance constraint leaves little room for placement decisions anyway); the buffer costs 37% running time (removing it saves 27%). The *incremental flush order* carries a large part of that value: replacing it by a single sort at flush time loses 27%, most on ISPD98 ($1.55\times$), whose $m/n \approx 1$ and small nets mean that the assigned fractions change continuously while the buffer drains, so a one-shot ranking is stale after the first few placements. The *home credit* is time-neutral and worth 27%; its effect is concentrated on SAT14 ($1.87\times$ without it), where the high m/n produces many identical clauses that become internal early: the share of nets taking the internal-net shortcut falls from 41% at $k = 2$ to 0.4% at $k = 2048$ (pooled over all nets of the tuning set, which its two large SAT instances dominate), and without the credit these nets leave no trace in the sketches, so the sweep pulls vertices out of perfectly settled clusters. The *placement cap* is a small effect (+0.9% overall, uniform across the families), visible on cells whose first pass fills blocks to $L_{\max}$. The *candidate restriction* is the one component that helps both objectives at once: switching it off costs 4.7% connectivity and 30% running time, with both penalties largest at large k ($1.12\times$ connectivity at $k = 512$) and concentrated on the dense nets of the sparse-matrix (SPM) family ($1.13\times$; **gupta3** averages 555 pins per net), where the occupied-block list per net is long and dominated by single-pin noise. The *large-net rules* are quality-neutral (1.01, within tie noise) and purely save time: $2.3\times$ on **gupta3** and $1.4\times$ on **msc10848** at $k = 2048$, and nothing elsewhere. The *final sketch sweep* is worth 14% overall and is what keeps the SAT family competitive ($1.38\times$ without it); the largest single-cell effect is E02F22 at $k = 128$, where the sweep improves connectivity by a factor of 2.1 while executing 63 283 moves in 20 rounds, the round cap, and it costs 13% running time. The *larger sketch* ($t = 5$ over $t = 3$) contributes 5.0%, concentrated on SAT14 ($1.12\times$ with $t = 3$) and on $k \geq 32$, where five tracked blocks disambiguate more of the candidates.

Build-Up Series. Table 5 shows the same components added one at a time in the order in which they entered the algorithm, starting from the basic algorithm of Section 3.1. The chain confirms the leave-one-out picture with one instructive difference: *buffering* with a one-shot sort improves the basic algorithm by only 3%, and it is the switch to the *incremental order* that produces the largest single step of the whole chain ($\times 0.80$), followed by the *home credit* ($\times 0.86$) and the *sweep* ($\times 0.91$). The build-up also shows the time structure: the basic algorithm runs in $0.78\times$ the time of the full configuration, *buffering* adds the largest cost, and the *candidate restriction* buys most of it back.

Sketch Size, Sweep, and Buffering Threshold. Table 6 isolates the one interaction worth showing, between the *sketch size* and the *sweep*. Without the sweep the two sketch sizes are within two percent of each other overall; the value of $t = 5$ only materializes once the sweep can exploit the two extra slots, and on SAT14, where sketches carry the most alternatives, the full sweep turns a 38% deficit into parity. A single sweep round recovers only about a third of the sweep’s total gain and three rounds about 80%, so the sweep’s value lies in iterating to its stopping rule rather than

in the first pass over the vertices. The *buffering threshold* θ completes the picture of the quality components: raising it to 0.9 buys another 1.9% connectivity at unchanged mean time but up to $2.1\times$ on single cells, and lowering it to 0.5 loses 2.4% for a 7% time saving; the default $\theta = 0.7$ trades a little of that quality for a bounded worst-case cell time.

Restreaming Series. Table 7 reports the restreaming series of Section 3.4, each pass followed by the sweep. The gains concentrate at $k \geq 8$ and on the circuit and SAT families (8 to 10% after three passes, 6 to 17% after ten), while the SPM family does not profit at all ($1.03\times$) and $k = 2$ degrades with more passes ($1.09\times$ at ten), where the rising balance penalty of the later passes pushes vertices out of two almost-settled halves. The second pass contributes the most (4.3%), and the gains do not dry up afterwards: the third pass adds 0.7%, five add 1.1% over three, and ten another 1.0% over five, so the series keeps improving to the end of the tested range at steeply growing cost. We carry two restreaming configurations into the following sections: $R = 3$ ($1.85\times$ the single-pass time) as the knee of the trade-off, where 5.0% of the 6.9% total gain is realized at 25% of the extra time, and $R = 10$ ($4.38\times$) as the upper end. The series also settles the sketch-accumulation question of Section 3.4: resetting the sketches between passes makes three-pass restreaming 9.2% worse at 5% more time, so accumulation is clearly right.

Buffer Capacity Series. Table 8 varies the capacity of the first-pass buffer around the default $B = 2^{14}$, from no buffer at all to 2^{20} nets. The quality is monotone in the capacity almost everywhere: no buffer costs 33%, 2^{10} still 21%, and 2^{12} 9%, while growing beyond the default gains another 1% at 2^{16} , 4% at 2^{18} and 8% at 2^{20} , at $1.06\times$, $1.20\times$ and $1.31\times$ the running time and up to $1.34\times$ the peak memory. The one exception beyond last-digit noise is $k = 2$, where 2^{16} is 7% worse than the default, whereas 2^{18} and 2^{20} are not. The default is a memory decision rather than a quality optimum: by the quality-time criterion alone, 2^{20} (8% at $1.31\times$ the time) would even beat three restreaming passes (5% at $1.85\times$, Table 7); $B = 2^{14}$ is within 1% of 2^{16} , which takes 6% more time and 14% more memory (with three passes within 0.4%), and as a constant it keeps the transient state independent of m , which Section 4.6 exploits. The constant is not small against the benchmark, though: 45 of the 310 instances (15%) have fewer than 2^{14} nets, so the buffer holds their entire net list and the first pass reduces to an in-memory ordering of it (18 of the 90 cells of this section); 130 instances have fewer than 2^{16} and 257 fewer than 2^{20} nets, so the largest capacity would buffer the whole input on 83% of the benchmark, whereas the default buffers 18% of the median instance (90 630 nets).

Configuration	Conn. / FREIGHT	$\times$ prev.	Time rel.
Basic algorithm (no components, $t = 3$)	1.19	–	0.78
+ buffering, one sort at flush	1.15	0.97	1.08
+ incremental flush order	0.92	0.80	1.10
+ candidate restriction	0.88	0.96	0.87
+ home credit	0.76	0.86	0.90
+ placement cap	0.76	1.00	0.90
+ final sketch sweep	0.69	0.91	1.00
+ sketch size $t = 5$	0.66	0.96	1.05
+ large-net rules (full configuration)	0.66	0.99	1.00

Table 5: Building the algorithm up component by component, in the order in which the components entered it: geometric mean connectivity relative to **FREIGHT-con** over the 90 cells (lower is better), the factor each step contributes relative to the previous row, and the running time relative to the full configuration.

		Connectivity rel.				Time rel.			
	Sketch size	0	1	3	20	0	1	3	20
All	$t = 3$	1.16	1.12	1.08	1.05	0.86	0.89	0.91	0.95
	$t = 5$	1.14	1.09	1.03	1.00	0.88	0.92	0.94	1.00
SAT14	$t = 3$	1.42	1.33	1.22	1.12	0.80	0.83	0.85	0.94
	$t = 5$	1.38	1.30	1.08	1.00	0.83	0.86	0.88	1.00

Table 6: Interaction of the sketch size t with the number of sweep rounds (0, 1, 3, 20; column headers): geometric mean connectivity and running time relative to the full configuration ($t = 5$, up to 20 rounds), over all 90 cells and over the SAT14 family alone. The series are separate runs; where they repeat a configuration of Table 4, the two differ by at most one unit in the last digit, which is the noise level.

4.2 Solution Quality

The component study of Section 4.1 used the fifteen tuning instances; from here on, every algorithm runs on all 310 instances of the benchmark at the six values of k , that is, on 1860 instance- k pairs, and Table 9 summarizes the evaluation. Among the streaming algorithms, HedgeStream computes the best partitions in geometric mean for both objectives: its connectivity improves over **Hashing** by 537% in geometric mean, compared with 322% for **FREIGHT-con**, 208–213% for the buffered **Min-Max** variants and 160–173% for **Min-Max-N2P** and **Min-Max-L5**, and its cut-net improves by 144% against 112% for **FREIGHT-cut** and 54–56% for the buffered variants. Measured directly against **FREIGHT**, HedgeStream’s connectivity is lower by a factor of 1.51 in geometric mean (bootstrap 95% interval [1.42, 1.60]; HedgeStream is better on 239 of the 310 instances, **FREIGHT** on 71, Wilcoxon $z = 12.0$; per-instance ratios are geometric means over the six values of k), and its cut-net by a factor of 1.15 ([1.10, 1.20]; 186 versus 124 instances, $z = 5.7$); the median per-instance ratios are 1.41 and 1.07. On the 295 instances that were not used for tuning the factors are 1.51 and 1.16, against 1.52 and 1.01 on the 15 tuning instances, so the connectivity result generalizes; the cut-net ratio of the tuning set understates the benchmark value because every parameter, including those of the cut-net configuration, was fixed on connectivity (the component study of Section 4.1 reports connectivity only), so that the cut-net configuration inherits parameters tuned for the other objective.

Since HedgeStream defers up to $B = 2^{14}$ nets while **FREIGHT** decides every vertex on arrival, Table 9 also lists HedgeStream without the buffer ($B = 0$), which has no lookahead at all: it loses 29% connectivity and 25% cut-net against the default at 0.68–0.70 times its time and 0.80 times its memory, but its connectivity is still better than **FREIGHT-con**’s by a factor of 1.17, and by 1.18

		Connectivity relative to one pass						Time
Passes	$k=2$	8	32	128	512	2048	All	rel.
2	1.00	0.97	0.95	0.96	0.91	0.95	0.96	1.45
3	1.03	0.96	0.94	0.95	0.89	0.93	0.95	1.85
5	1.05	0.95	0.92	0.95	0.88	0.90	0.94	2.61
10	1.09	0.94	0.91	0.92	0.86	0.88	0.93	4.38

Table 7: Restreaming on the fifteen instances of Table 3: geometric mean connectivity and running time of R passes, each pass followed by the sweep, relative to a single pass followed by the sweep.

Capacity B	Connectivity relative to the default $B = 2^{14}$						Time	Mem.	
	$k=2$	8	32	128	512	2048	All	rel.	rel.
<i>One pass:</i>									
no buffer	1.90	1.56	1.42	1.20	1.08	1.01	1.33	0.73	0.83
2^{10}	1.63	1.43	1.25	1.11	1.00	0.98	1.21	0.89	0.82
2^{12}	1.34	1.13	1.09	1.06	0.97	0.98	1.09	0.89	0.88
2^{14}	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
2^{16}	1.07	0.98	0.98	0.99	0.96	0.97	0.99	1.06	1.14
2^{18}	1.00	0.88	0.92	1.00	1.01	0.98	0.96	1.20	1.21
2^{20}	0.95	0.80	0.88	0.97	0.95	0.98	0.92	1.31	1.34
<i>Three passes:</i>									
no buffer	1.89	1.54	1.34	1.15	1.05	1.00	1.30	0.93	0.82
2^{10}	1.61	1.30	1.20	1.06	0.98	0.98	1.17	1.00	0.85
2^{12}	1.34	1.10	1.08	1.03	0.97	0.98	1.08	0.98	0.86
2^{14}	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
2^{16}	1.04	0.97	0.99	0.98	1.01	0.99	1.00	1.03	1.14
2^{18}	0.99	0.88	0.93	0.97	1.00	0.98	0.96	1.11	1.19
2^{20}	0.95	0.80	0.89	0.97	1.00	0.98	0.93	1.18	1.29

Table 8: Buffer capacities on the fifteen instances of Table 3: geometric mean connectivity per k and over all 90 cells, running time, and peak resident memory, each relative to the default $B = 2^{14}$ with the same number of passes. The row 2^{14} is the default itself and reads one by construction.

on the 265 instances whose net list exceeds the default buffer; **FREIGHT-con** is ahead by 4% only on the 53 instances with more than a million nets, and **FREIGHT-cut** by 9% in cut-net. The lead in connectivity therefore does not rest on the buffer, whose capacity is a constant, and we do not evaluate the quality of the buffer-free configuration any further.

The three-pass configuration fixed in Section 4.1 raises the improvement over **Hashing** to 555% in connectivity and 156% in cut-net; the objectives fall by another 2.7% and 4.7% relative to the single pass; the gain is smaller than the 5.0% the tuning set predicted for connectivity, because the full benchmark contains many more of the sparse-matrix instances on which restreaming gains little. Ten passes continue this pattern only for the cut-net metric (161%) and for large k (a connectivity improvement of 296% against 287% for three passes at $k \geq 512$): in the connectivity aggregate over all pairs they fall back to 548%, below three passes, because the degradation at $k = 2$ that Table 7 shows on the tuning set ($1.09\times$ at ten passes) carries over to the benchmark ($1.11\times$). The tuning set also overpredicts the ten-pass gain on the SAT family: it promised 6 to 17% (Section 4.1), whereas on the benchmark ten passes leave the SAT instances 4.5% *worse* in connectivity and $k = 8$ marginally worse ($1.01\times$), while they improve the circuits by 15% and the sparse matrices by 3%. The upper end of the quality-time trade-off is therefore metric- and k -dependent: ten passes buy the best cut-net and the best large- k connectivity, three passes the best overall connectivity. Per pair, three passes improve the connectivity on 71% of the pairs and worsen it on 27%, ten passes on 66% and 33%.

Table 10 adds bootstrap confidence intervals to the improvements of Table 9. The intervals of HedgeStream and **FREIGHT** are disjoint for connectivity ($[469, 616]$ against $[281, 368]$) and overlap for cut-net ($[124, 164]$ against $[98, 128]$), where the paired Wilcoxon test above decides in HedgeStream’s favor ($z = 5.7$); the intervals of the three buffered **Min-Max** variants overlap almost entirely, and those of **Min-Max-N2P** and **Min-Max-L5** overlap.

Algorithm	Impr. over Hashing [%]		Impr. at $k \geq 512$ [%]		Infeas. [%]	ns per pin	Slow-down
	conn	cut	conn	cut			
HedgeStream-con	537	–	268	–	0.0	72.1	12.1
HedgeStream-cut	–	144	–	36	0.0	74.8	12.6
HedgeStream-con (3 passes)	555	–	287	–	0.0	140.1	22.8
HedgeStream-cut (3 passes)	–	156	–	42	0.0	156.1	24.6
HedgeStream-con (10 passes)	548	–	296	–	0.0	390.4	56.7
HedgeStream-cut (10 passes)	–	161	–	46	0.0	429.0	61.8
HedgeStream-con (no buffer)	394	–	244	–	0.0	51.2	8.2
HedgeStream-cut (no buffer)	–	96	–	31	0.0	53.8	8.8
FREIGHT-con	322	–	210	–	0.0	27.6	4.1
FREIGHT-cut	–	112	–	41	0.0	27.2	4.0
REF_RLX	213	56	150	18	5.2	575.9	94.4
REF_RLX_SV	210	55	148	18	4.0	507.1	78.8
REF	208	54	146	18	3.8	517.4	84.8
Min-Max-N2P	173	45	120	14	3.4	300.8	50.0
Min-Max-L5	160	44	104	14	3.5	94.3	15.5
Min-Max	79	26	38	5	5.8	1438.3	346.3
Hashing	0	0	0	0	21.9	8.9	1.0
SIGHP	398	126	260	45	0.1	350.7	64.4
HYPE	274	85	187	32	5.9	563.5	92.1
PaToH-con	1578	–	506	–	0.1	1660.8	249.3
PaToH-cut	–	584	–	114	0.0	1313.6	197.2

Table 9: All 21 configurations on 1 860 instance- k pairs: geometric mean improvement over **Hashing** in connectivity and cut-net over all pairs and over the pairs with $k \in \{512, 2048\}$; fraction of pairs whose majority of runs is infeasible; median running time per pin in nanoseconds; geometric mean slowdown over **Hashing** (memory: Section 4.4). The in-memory algorithms **SIGHP**, **HYPE** and **PaToH** read the whole hypergraph and are reference points, not competitors in the streaming setting. **Hashing** serves as the common denominator only; at $k \geq 512$ it violates the balance constraint on part of the pairs (its infeasibility column), and every head-to-head claim of the text rests on the direct ratios between feasible algorithms, not on this column. Pairs with an exact-zero objective enter via the zero-replacement rule of the text (eight for HedgeStream and **FREIGHT** together; more for **PaToH** and the restreaming configurations).

Figure 1 shows how the comparison depends on k . The connectivity advantage over **FREIGHT-con** is a factor of 2.2 at $k = 2$, 1.65 and 1.60 at $k = 8$ and 32, 1.47 at $k = 128$, 1.28 at $k = 512$ and still 1.10 at $k = 2048$, so HedgeStream leads at every k of the grid. For the cut-net the two algorithms cross: HedgeStream-cut is ahead by factors of 1.57 to 1.07 for $k \leq 128$, and **FREIGHT-cut** is ahead by 1% at $k = 512$ and by 5% at $k = 2048$, where the cut-net metric charges a net once no matter how many blocks it spans, and the exact per-net state of the vertex-stream model has its greatest value. The gap is largest at $k = 2$ because **FREIGHT** has to place the first vertices of every net with nothing but the identifier hash to go on while HedgeStream buffers them; **SIGHP** is behind HedgeStream in connectivity at every k except $k = 2048$, where it is ahead by 2%, its only lead on the connectivity grid; in cut-net it leads at $k \geq 128$, by 1 to 7%.

Table 11 breaks the comparison down by family. HedgeStream leads every streaming baseline in connectivity on all three families, with the largest margin on the sparse matrices (778% versus 438% for **FREIGHT-con**), which make up 62% of the benchmark, and the smallest on the SAT instances (280% versus 198%); in cut-net it leads on the circuits (124% versus 78%) and the matrices (168%

Algorithm	Connectivity impr. [%]	Cut-net impr. [%]
HedgeStream-con	537 [469, 616]	–
HedgeStream-cut	–	144 [124, 164]
HedgeStream-con (3 passes)	555 [483, 635]	–
HedgeStream-cut (3 passes)	–	156 [135, 177]
HedgeStream-con (10 passes)	548 [477, 628]	–
HedgeStream-cut (10 passes)	–	161 [140, 183]
FREIGHT-con	322 [281, 368]	–
FREIGHT-cut	–	112 [98, 128]
REF_RLX	213 [197, 232]	56 [50, 61]
REF_RLX_SV	210 [194, 227]	55 [50, 60]
REF	208 [192, 226]	54 [49, 59]
Min-Max-N2P	173 [161, 188]	45 [41, 49]
Min-Max-L5	160 [148, 173]	44 [40, 48]
Min-Max	79 [72, 86]	26 [24, 29]
SIGHP	398 [358, 442]	126 [109, 144]
HYPE	274 [243, 310]	85 [75, 95]
PaToH-con	1578 [1360, 1807]	–
PaToH-cut	–	584 [500, 679]

Table 10: Geometric mean improvement over **Hashing** over all 1 860 pairs with 95% confidence intervals from a bootstrap over the 310 instances (1 000 replicates).

versus 111%), whereas **FREIGHT-cut** is ahead on the SAT instances (123% versus 106%). The SAT instances are the hardest for a sketch of five slots: they have few vertices and many nets, a variable is seen by hundreds of clauses spread over many blocks, and the sweep of Section 3.3 can only move a vertex to a block its sketch tracks. The in-memory reference points put the streaming results into perspective. **PaToH** computes connectivities lower than HedgeStream’s by a factor of 2.6 overall, from 5.6 at $k = 2$ down to 1.5 at $k = 2048$, and cut-nets lower by 2.8; this is the price of a single sequential pass over the input. **HYPE**, the fast in-memory partitioner, is behind HedgeStream by a factor of 1.70 in connectivity and 1.32 in cut-net. **SIGHP**, which reads the whole hypergraph into memory, is behind by 1.28 in connectivity overall ($[1.18, 1.38]$, $z = 5.7$) and by 1.08 in cut-net, a marginal difference ($[1.00, 1.16]$, $z = 2.2$); the aggregate hides a split by family, since **SIGHP** is on par with HedgeStream in connectivity on the circuits but ahead in cut-net there (173% versus 124%), and clearly better on the SAT instances (335% versus 280% in connectivity, 224% versus 106% in cut-net), while HedgeStream is better on the sparse matrices by a wide margin (778% versus 450%).

The aggregates above hide how often each algorithm is close to the best; Figure 2 shows this with performance profiles for both objectives, with all algorithms in one pool so that the in-memory partitioners set the reference. **PaToH** computes the best connectivity on 93% of the pairs and the best cut-net on 96%, and no other algorithm comes close to it at $\tau = 1$. Beyond $\tau = 1$ HedgeStream is the curve that rises fastest of the remaining ones: it is within a factor of two of the best connectivity on 54% of the pairs, against 36% for **FREIGHT-con**, 43% for **SIGHP** and 26% for **HYPE**, and within a factor of two of the best cut-net on 52%, against 49% for **FREIGHT-cut** and 50% for **SIGHP**. Restricted to the streaming algorithms (a count at $\tau = 1$ that the figure does not show), the ten-pass HedgeStream-con computes the best connectivity on 46% of the pairs, **FREIGHT-con** on 20%, the one-pass HedgeStream-con on 18%, the three-pass on 14% and the three buffered **Min-Max**

Hashing	10.51	8.89	7.91	6.71	4.50	3.01
MM-N2P	4.28	2.73	2.32	2.13	1.85	1.50
RRS	3.84	2.43	2.02	1.83	1.62	1.36
HYPE	2.93	1.91	1.70	1.56	1.36	1.21
FREIGHT	2.17	1.65	1.60	1.47	1.28	1.10
SIGHP	2.10	1.37	1.23	1.19	1.07	0.98
PaToH	0.18	0.26	0.36	0.48	0.57	0.65
	2	8	32	128	512	2048

Number of blocks k

(a) Connectivity.

Hashing	8.15	3.29	2.30	1.83	1.48	1.26
MM-N2P	3.32	2.01	1.64	1.45	1.26	1.13
RRS	2.98	1.84	1.51	1.36	1.21	1.10
HYPE	2.28	1.46	1.27	1.17	1.07	1.00
FREIGHT	1.57	1.26	1.15	1.07	0.99	0.95
SIGHP	1.63	1.09	1.00	0.99	0.95	0.93
PaToH	0.14	0.23	0.34	0.46	0.58	0.70
	2	8	32	128	512	2048

Number of blocks k

(b) Cut-net.

Figure 1: Objective of each algorithm relative to HedgeStream, geometric mean over the 310 instances per k : a cell above one (orange) means the algorithm computes worse partitions than HedgeStream on that k , a cell below one (blue) better ones; the color encodes $\log_2$ of the ratio, clipped to ± 2.5 (**Hashing** at $k = 2$ exceeds the scale); HedgeStream is the implicit reference row with ratio one. Rows are ordered by the connectivity ratio. The **Hashing** row inherits the caveat of Table 9. Abbreviations here and in the following figures: RRS = REF_RLX_SV, MM = Min-Max, MM-N2P = Min-Max-N2P, MM-L5 = Min-Max-L5, HS = HedgeStream, HS-3p and HS-10p (HS-con-3p and HS-con-10p) = HedgeStream with three and ten passes, FR = FREIGHT, Hash = Hashing.

variants together on 6%; in cut-net the ten-pass HedgeStream-cut is the best on 40%, **FREIGHT-cut** on 32%, the three-pass on 17%, the one-pass on 15% and the buffered variants together on 12% (ties count for each algorithm). The share of the one-pass HedgeStream-con is depressed by its own

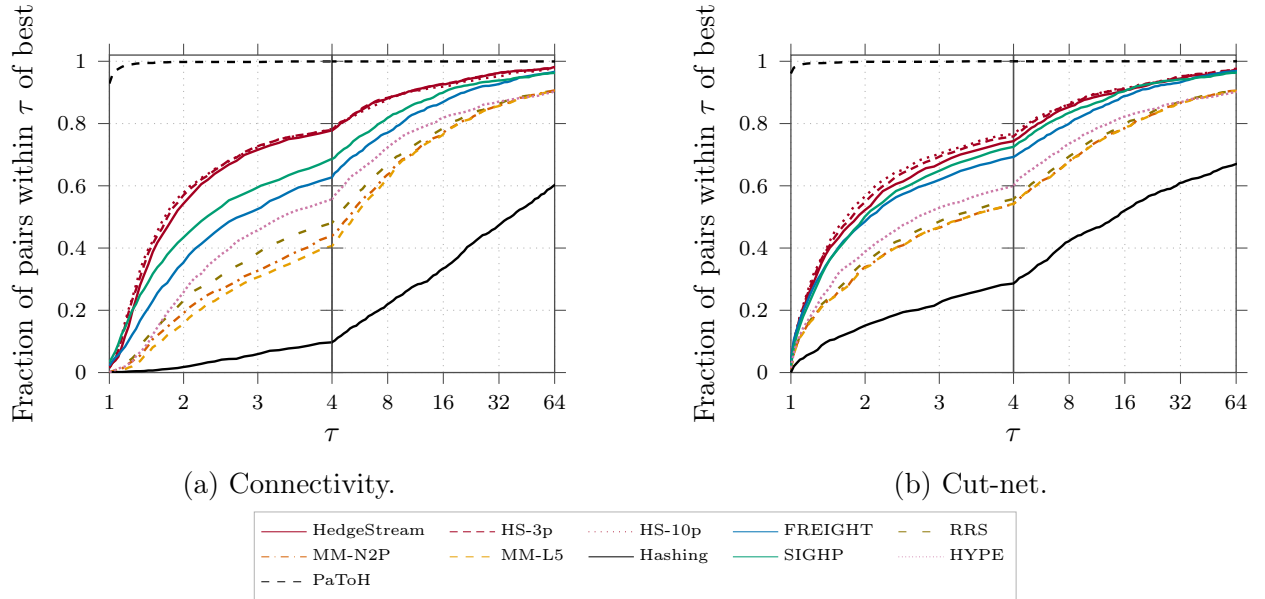


Figure 2: Performance profiles over all 1 860 instance- k pairs, with the in-memory partitioners included; a pair whose majority of runs is infeasible for an algorithm counts as unsolved for that algorithm. REF and REF_RLX (not plotted) coincide with REF_RLX_SV; Min-Max lies between Min-Max-L5 and Hashing.

Algorithm	Connectivity impr. [%]			Cut-net impr. [%]		
	ISPD98	SPM	SAT14	ISPD98	SPM	SAT14
HedgeStream	266	778	280	124	168	106
FREIGHT	124	438	198	78	111	123
REF_RLX	135	263	147	53	42	87
Min-Max-N2P	110	215	119	43	34	70
SIGHP	270	450	335	173	85	224
HYPE	135	391	142	73	86	86
PaToH	980	1670	1537	653	410	1081

Table 11: Geometric mean improvement over **Hashing** by instance family, over all six values of k (18 ISPD98 circuits, 192 sparse matrices, 100 SAT instances). The **Hashing** denominator inherits the caveat of Table 9.

restreaming variants, which take most of its best results; in a direct comparison of the one-pass configurations, HedgeStream computes a better connectivity than **FREIGHT** on 71% of the pairs and a worse one on 28%, and a better cut-net on 54% against 43% (the rest are ties). The **Hashing** curve saturates below one because 22% of its pairs are infeasible, and the **Min-Max** curves end 3 to 6 points below one for the same reason.

4.3 Running Time

Figure 3 shows the running time per pin, that is, the time of the partitioning kernel divided by the number of pins of the instance. Every tool reports its own kernel time, measured after the input has been parsed into memory and before the partition is written; we verified the convention in the sources of every baseline (**FREIGHT** and **Hashing** with `-ram_stream`, the **Min-Max** family on a pre-parsed compressed representation, **SIGHP** on its memory-mapped edge list, whose page faults therefore fall into its kernel time) and HedgeStream times its pass, the buffer flush and the sweep.

Hashing is the fastest algorithm on 99.6% of the pairs, at a median of 9 ns per pin, followed by **FREIGHT** at 27 ns and HedgeStream at 72–75 ns; the geometric mean slowdown over **Hashing** is 4.0–4.1 for **FREIGHT** and 12.1–12.6 for HedgeStream, so HedgeStream is 2.9 to 3.1 times slower than **FREIGHT** per pin (one-pass configurations paired by objective). The next group starts at **Min-Max-L5** with 94 ns and reaches 300 ns for **Min-Max-N2P**, 507–576 ns for the buffered variants and 1.4 μ s for **Min-Max**. The in-memory partitioners are slower than every streaming algorithm of the Pareto frontier, but overlap with the slower streaming configurations, **SIGHP** undercutting the buffered **Min-Max** variants and the ten-pass HedgeStream in median: **SIGHP** takes 351 ns per pin in median, **HYPE** 564 ns and **PaToH** 1.3–1.7 μ s, and their geometric mean slowdowns over **Hashing** are 64, 92 and 197–249 (all other factors in this section are ratios of medians unless stated otherwise).

The right panel shows the dependence on k : **FREIGHT** and **Hashing** are flat, since a vertex is charged $O(k)$ once and the cost is dominated by the pass over the pins, whereas HedgeStream’s median grows from 46 ns at $k = 2$ to 122 ns at $k = 2048$, a factor of 2.7, so that HedgeStream is 1.9 times slower than **FREIGHT** at $k = 2$ and 4.5 times at $k = 2048$ in geometric mean. The growth is not a k -dependent loop, since the candidate restriction of Section 3.1 bounds the work per pin, but the share of pins on the cheap paths: on **ibm16**, for instance, the share of nets that take the internal-net shortcut falls from 38% at $k = 2$ to 12% at $k = 2048$, so that more pins reach the sketch update and the move evaluation; on **Emilia_923** the shortcut share falls from 73 to

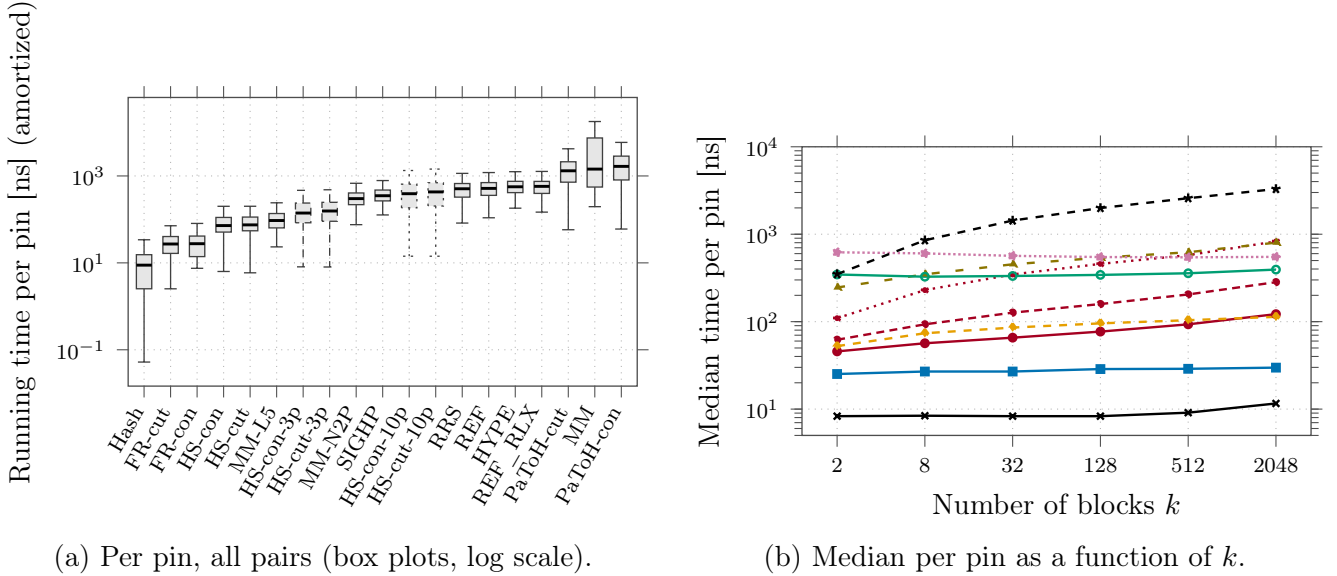


Figure 3: Running time per pin over all 1860 pairs (left, sorted by median) and its dependence on k (right). Conversion of the input into the format of an algorithm is not included. Hashing touches only the vertices, so its time per pin falls below one clock cycle on instances with many pins per vertex.

22% between $k = 2$ and $k = 512$ with the same consequence. Without the candidate restriction, when every occupied block is a candidate, the median over the tuning set at $k = 2048$ rises from 102 to 197 ns and the slowest cell from 348 ns to $9.1 \mu\text{s}$, while $k = 2$ is unaffected (Section 4.1). PaToH’s time grows by a factor of 9 from $k = 2$ to $k = 2048$, so that at large k it is 29 times slower than HedgeStream in geometric mean, whereas SIGHP and HYPE are flat in k and their factor over HedgeStream shrinks from 8 and 14 at $k = 2$ to 4.4 and 4.5 at $k = 2048$ (geometric means over the pairs at each k).

The times of the streaming algorithms include the buffer flush of the first pass; the sweep of Section 3.3 accounts for about 13% of HedgeStream’s time on the tuning set (Table 4), more on the small instances, whose whole run takes a tenth of a second, than on the large ones. The three-pass configuration runs at a median of 140–156 ns per pin, a geometric mean slowdown of 22.8–24.6 over Hashing and 1.9 to 2.0 times the single pass in geometric mean, consistent with the 1.85 measured on the tuning set: the restreaming passes are cheaper than the first where the internal-net pre-test of Section 3.4 skips a large share of the input, which is the case at small k : three passes cost 1.4 times the single pass at $k = 2$ and 2.3 times at $k = 2048$, ten passes 2.6 and 6.8 times (connectivity configuration; the cut-net configuration lies 7 to 10% above these factors), so the restreaming gains, which also concentrate at large k , are bought at the highest relative price there. The peak memory of the restreaming configurations is identical to the single pass, since restreaming adds no state. Ten passes run at a median of 390–429 ns per pin, a slowdown of 57–62 over Hashing and 4.7 to 4.9 times the single pass in geometric mean, again with identical peak memory.

Figure 4 puts the two objectives of this section and the previous one together and plots quality against time. Among the streaming algorithms the Pareto frontier consists of Hashing, FREIGHT-con and HedgeStream-con in its buffer-free ($B = 0$), its one-pass and its three-pass configurations: every other streaming algorithm is slower than HedgeStream and computes worse

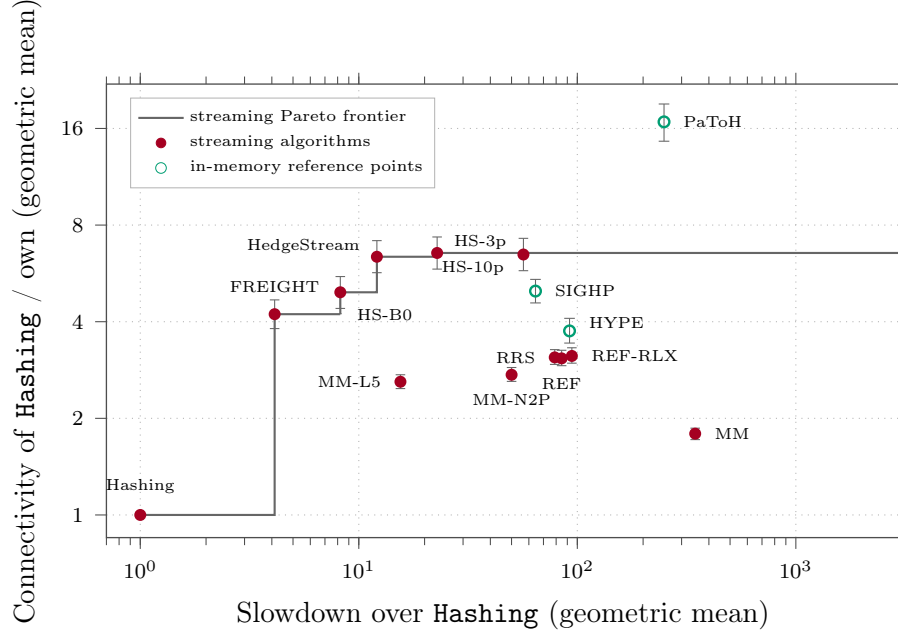


Figure 4: Time–quality trade-off over all 1860 pairs: geometric mean slowdown over **Hashing** against the geometric mean ratio of **Hashing**’s connectivity to the algorithm’s own, with 95% bootstrap intervals; up and to the left is better; the **Hashing** reference inherits the caveat of Table 9.

partitions, by factors of 2.0 to 3.6 in connectivity at 1.3 to 29 times the time. The ten-pass configuration lies below the frontier: its connectivity aggregate falls behind the three-pass one at 2.5 times its time. The in-memory partitioners lie to the right of the streaming Pareto frontier; **SIGHP** and **HYPE** are below **HedgeStream**’s line, and only **PaToH** is above it, at 21 times **HedgeStream**’s time.

4.4 Memory Usage

Figure 5 shows the peak resident memory of every algorithm relative to **PaToH**’s on the same pair, so that the size of the instance cancels out. All algorithms hold their input in memory for this experiment, so the footprint of a streaming algorithm is the sum of its persistent state and its copy of the pin lists, and the differences between the streaming algorithms reflect both the state and the width of the input format: **FREIGHT** holds the input as binary pin lists with 64-bit entries plus a per-net vector, **HedgeStream** as 32-bit pin lists, and the **Min-Max** variants parse it into a compact structure, so the comparison between **HedgeStream** and **FREIGHT** is a comparison of the two tools as they run, not of their state alone. **Hashing** keeps no state beyond the partition and is left out of the memory comparison. **HedgeStream** has the smallest footprint of all compared algorithms: 29% of **PaToH**’s memory in geometric mean over all pairs (24 MiB in median in absolute terms), against 40% for **FREIGHT**, 31–35% for **Min-Max-N2P**, **Min-Max-L5** and the buffered variants, 58% for **SIGHP** and 153% for **HYPE**; **HedgeStream** therefore needs 1/1.36 of **FREIGHT**’s memory, that is, 26% less; since the input copy is four fifths of both footprints, this factor mostly measures the 64-bit against the 32-bit pin lists, and the comparison of the persistent state is the one of Section 4.6. On the 37 instances with at least ten million pins, where the state matters more than fixed overheads, the shares fall to 19% for **HedgeStream** and 31% for **FREIGHT**, so the factor over **FREIGHT** grows to 1.7; the in-memory partitioners **SIGHP** and **HYPE** need 50% and 156% of **PaToH**’s memory there, that

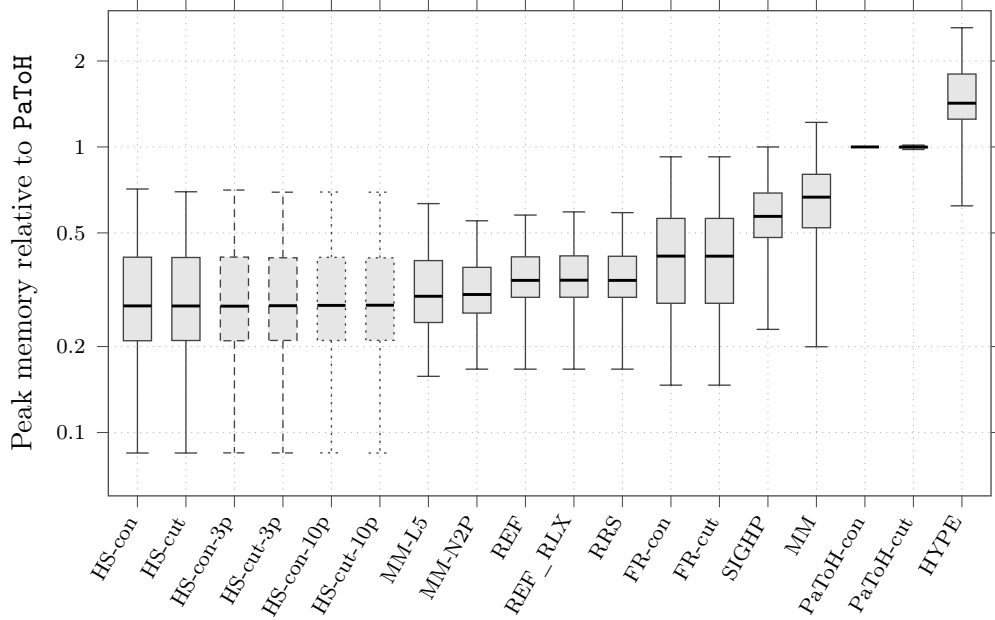


Figure 5: Peak resident memory over all 1 860 pairs, relative to PaToH-con on the same pair, sorted by median (box plots, log scale). Every algorithm streams or reads its input from internal memory, so the footprint includes the pin lists in the algorithm’s own format. The boxes summarize the per-pair ratios; the geometric means quoted in the text differ from the medians of the boxes.

is 2.7 and 8.4 times HedgeStream’s (2.0 and 5.3 times over all pairs), and PaToH itself 5.4 times (3.4 over all pairs). The persistent state of Section 3.6, 49 bytes per vertex at $\beta = 2$ and 41 at $\beta = 1$, is a fifth of HedgeStream’s footprint on the large instances in median and at most 45%; the rest is the in-memory copy of the input, which a disk-streaming driver does not need, so that the persistent state, plus the first-pass buffer of at most 2^{14} nets, is what remains against FREIGHT’s $\Theta(m)$ per-net state; Section 4.6 measures that setting, and the memory numbers above are those of the tools as run here. Streamed from disk under the protocol of Section 4.6, the whole benchmark confirms this arithmetic (all 310 instances at seed 1; every run completes and returns exactly the objective value of the in-memory run): without the input copy, HedgeStream’s peak falls to 0.42 of its in-memory value in geometric mean over the 1 860 pairs (10 MiB in median, at most 506 MiB) and FREIGHT’s to 0.18 (4 MiB in median, at most 175 MiB). The ordering therefore reverses: with the copies gone, FREIGHT’s few bytes of per-net state undercut HedgeStream’s 49 bytes per vertex on 74% of the pairs and by a factor of 1.69 in geometric mean, and Section 4.6 traces when the per-vertex state wins instead.

4.5 Stream-Order Robustness

All streams of this evaluation arrive in file order, whose locality an algorithm might exploit. We therefore repeated the single-pass comparison on the whole benchmark, all 310 instances and all six values of k , under uniformly random stream orders: three permutations of the hyperedge order of the hMetis file for HedgeStream and, analogously, three of the vertex order of the vertex-stream file for FREIGHT (net identifiers unchanged, and every run evaluated against exactly the file the tool consumed): 1 860 cells, four orders, and three configurations (HedgeStream’s one- and three-pass connectivity configurations and FREIGHT-con), 22 320 runs in total. Every run completed,

and both tools prefer the file order, but **FREIGHT** depends on it twice as much: a random order raises HedgeStream’s connectivity to 1.29 times its file-order value in geometric mean over the 5 559 permuted cells with a nonzero file-order reference, **FREIGHT**’s to 1.63 times, so the head-to-head connectivity ratio widens from 1.50 in file order to 1.90 under permutation (the file-order value differs from the 1.51 of Section 4.2 because these runs use a single repetition and exclude the zero-reference cells); the advantage of Section 4.2 is a property of the algorithm, not of the input order. The sensitivity falls with k for both tools ($1.94\times$ and $2.17\times$ at $k = 2$, 1.03 and 1.32 at $k = 2048$); it is uniform across the families for HedgeStream (1.25 to 1.36) but concentrated on the sparse matrices for **FREIGHT** (1.97, against 1.03 on the circuits); and three restreaming passes do not recover the order damage ($1.27\times$ against the single pass’s $1.29\times$). HedgeStream’s smaller sensitivity is what the prioritized buffer is built for: it defers exactly the nets that a shuffled stream leaves under-informed and flushes them best-informed first, so the buffer reconstructs locally some of the order that the permutation destroyed. On the seven cells per tool whose file-order connectivity is zero, excluded from the ratios, every permutation leaves HedgeStream at zero, whereas **FREIGHT** loses the zero on most of them (to connectivity 1 to 7 on **Pd_rhs**, to thousands on **G67**).

4.6 The True Streaming Setting

All experiments so far hold the input in memory, apart from the footprint comparison of Section 4.4. Table 12 reports HedgeStream and **FREIGHT** in the setting the streaming model is about: the five largest benchmark instances (up to 9.8 M vertices and 114 M pins) and, beyond the benchmark, instances of up to 1.95 billion pins that close the section are read from a local NVMe disk, one file pass per streaming pass; every run completes and its peak memory is measured, and the times are the minimum of two repetitions, with the operating system’s page cache left in place between repetitions and after staging the file, so the times measure the streaming computation with warm-cache reads rather than cold-disk I/O, for both tools alike; the hMetis files measure 0.3 to 0.8 GB on the five benchmark instances and 7.0 to 17.2 GB on the four giants, so every input is cache-resident on the machine of Section 4. Every run, including all four giants, completed under a 10 GiB address-space limit imposed on the process, so the footprints of Table 12 are peaks of runs that actually fit a small-memory machine. Four observations carry the section.

First, both footprints are flat in k and small, and **FREIGHT**’s is the smaller one. On the five benchmark instances, HedgeStream peaks between 60 MiB (**Emilia_923**) and 506 MiB (**wb-edu**, whose 9.8 M vertices cost 49 bytes each in per-vertex state alone), 178 MiB in median over the 30 instance- k cells and a quarter of its own in-memory peak; its footprint is set by n and hardly moves with k . **FREIGHT** peaks between 18 and 175 MiB, equally independent of k , and stays 1.6 to 3.8 times below HedgeStream on the same cell, 2.8 times in geometric mean: with $m \leq n$ on all five instances, its $\Theta(m)$ state costs 8 to 17 bytes per net and vertex against HedgeStream’s 49 bytes per vertex, so on this benchmark the per-net state is the cheaper one, and the last paragraph of this section returns to when it is not.

Second, the streamed partitions are the in-memory partitions: for every configuration the connectivity from disk equals the in-memory value exactly and the streaming reader feeds the same nets in the same order, so the evaluation of the previous sections transfers to the streaming setting unchanged; on these five instances HedgeStream’s connectivity is better than **FREIGHT-con**’s by factors of 1.6 to 3.3 on four of them and worse by 0.83 on **hollywood-2009**, where **FREIGHT-con** is ahead at $k \geq 128$.

Third, parsing narrows the wall-clock gap without closing it. In memory, HedgeStream is three times slower than **FREIGHT** per pin (Section 4.3); from disk, reading and parsing dominate both wall

Instance	$n/10^3$	$m/10^3$	Pins/ 10^3	HedgeStream		FREIGHT-con		Conn.
				MiB	ns/pin	MiB	ns/pin	FR/HS
hollywood-2009	1 140	1 140	113 891	74	105.4	22	34.8	0.83
nlpkt120	3 542	3 542	96 846	186	92.8	58	47.4	3.27
circuit5M	5 558	5 558	59 524	319	99.0	175	82.9	1.63
wb-edu	9 846	6 920	57 157	506	137.1	134	124.5	2.57
Emilia_923	923	923	41 005	60	75.5	18	38.7	1.60
webbase-2001	118 142	90 482	1 019 903	6900	129.9	1649	94.7	2.03
it-2004	41 292	36 023	1 150 725	2520	98.5	700	48.7	1.40
sk-2005	50 636	43 733	1 949 413	3060	95.3	1129	39.8	3.26
sat_nurikabe_p18	28 987	351 665	810 593	1470	147.7	2912	60.8	1.76

Table 12: The five largest benchmark instances and the four larger instances of the last paragraph streamed from local disk: number of vertices, nets and pins in thousands, peak resident memory (RSS) in MiB (maximum over the six values of k) and wall-clock time of one pass including I/O (HedgeStream: the single pass plus the sweep; FREIGHT: its connectivity configuration) at $k = 32$, the middle of the grid, in nanoseconds per pin; connectivity of FREIGHT-con relative to HedgeStream (values above one mean that HedgeStream computes the better partition), in geometric mean over the six values of k ; the protocol, the equality of the streamed with the in-memory partitions, and FREIGHT’s conversion cost, not included in these times, are discussed in the text.

clocks (HedgeStream’s is twice its kernel time, FREIGHT’s 3.5 times its own), and the gap shrinks: at $k = 32$ a HedgeStream pass takes 76 to 137 ns per pin against 35 to 125 ns for FREIGHT-con; over the 30 cells a pass costs 1.1 to 3.5 times FREIGHT’s, 1.8 times in geometric mean, with both readers as shipped; a faster reader for FREIGHT is conceivable, but optimizing it is not the subject of this paper, and the transposition below is a cost of the model, not of the reader.

Fourth, before FREIGHT sees a single vertex, the hMetis input must be transposed to the vertex-stream format. We charge no conversion time or memory to FREIGHT in this table, since the transposition can itself be organized as a streaming computation, but the cost that no converter can remove in the streaming model remains: at least two further passes over the input and a second copy on disk, in the same text encoding and 27 to 30% larger than the original on the five benchmark instances (23 to 36% on the giants), before the first pass can start. HedgeStream consumes the hMetis file directly and pays none of this.

Table 12 also lists four larger instances: the web hypergraphs webbase-2001, it-2004 and sk-2005 from the SuiteSparse (formerly University of Florida) collection [14] in the row-net model of Section 4 (empty rows dropped), with 118, 41 and 51 M vertices and 1.02, 1.15 and 1.95 billion pins, and the SAT formula sat_nurikabe_p18 from the Global Benchmark Database in the primal model of the SAT14 family (29 M variables, 352 M clauses, 0.81 billion pins), whose nets outnumber its vertices twelvefold, streamed under the same protocol; no in-memory run exists for them, and every cell completed. Three findings mark this scale.

First, the memory gap of the benchmark widens. HedgeStream’s footprint follows n : its persistent state costs 49 bytes per vertex (Table 1), that is, 5.4, 1.9 and 2.3 GiB on the three web hypergraphs, which runs with the first-pass buffer disabled confirm within one percent, and the buffer raises the measured peaks to 6.7, 2.5 and 3.0 GiB; FREIGHT’s follows $n + m$ at a few bytes per net and per vertex, 1.6, 0.7 and 1.1 GiB, 0.24 to 0.37 of HedgeStream’s. With $m < n$ on all three web hypergraphs the per-vertex state is the larger one, and the memory argument of Section 3.6

is an argument about $m \gg n$: at 49 bytes per vertex against a few bytes per net, the per-vertex state wins only once nets outnumber vertices several times over. `sat_nurikabe_p18` is that case, measured: with $m = 12n$, `FREIGHT` peaks at 2.8 GiB, within a percent of the $8(n+m)$ term, against HedgeStream’s 1.4 GiB, so the ordering of the web hypergraphs flips and HedgeStream streams the instance in half of `FREIGHT`’s memory.

Second, HedgeStream’s pass takes 1.4, 2.0, 2.4 and 2.4 times `FREIGHT`’s wall-clock time. Third, at this scale the transposition itself has to stream: the converter shipped with `FREIGHT` builds the hypergraph in memory and needs 31 GiB on `webbase-2001`, 27 GiB on `it-2004`, 44 GiB on `sk-2005` and 32 GiB on `sat_nurikabe_p18`, so entering the vertex-stream model on inputs of this size requires a streaming converter; the file the transposition produces is 1.2 to 1.4 times the size of the original. The transposition is no rounding error in the wall clock either: the shipped converter takes 226 ns per pin on `sk-2005`, where the pass-time gap between the two tools at $k = 32$ is 55 ns per pin, and 374 ns per pin on `sat_nurikabe_p18` against a gap of 87 ns, so in a load-and-go pipeline the conversion alone outweighs `FREIGHT`’s faster passes several times over. The partitions favor HedgeStream: its connectivity is better than `FREIGHT-con`’s by factors of 2.0, 1.4, 3.3 and 1.8 in geometric mean over k (2.6, 1.0, 2.1 and 3.8 at $k = 2, 1.8, 1.4, 4.0$ and 1.1 at $k = 2048$). On inputs of this size `FREIGHT` thus retains the faster pass throughout and the smaller footprint as long as vertices outnumber nets; the case for the hyperedge-stream model rests on the better partitions, on the memory once nets take over, and on starting immediately, with no transposition and no second copy of the input.

5 Conclusion

We presented HedgeStream, a balanced hypergraph partitioner for the hyperedge-stream model that consumes hMetis-format inputs in a single sequential pass, with no transposition and no copy of the input. Its persistent state is dominated by one fixed-capacity Misra–Gries sketch per vertex, so the memory grows with the number of vertices plus an additive term in the number of blocks, apart from the transient first-pass buffer, and the quality that this approximation gives up is recovered by prioritized buffering with an incremental flush, a final sketch sweep and optional restreaming. The component study identifies buffering, the home credit and the sweep as the components that carry most of the quality, the candidate restriction on dense nets as the one component that improves quality and running time at once, and three restreaming passes as the knee of the quality–time trade-off, beyond which, on the full benchmark, further passes pay only for the cut-net metric and large k .

On the 310 benchmark hypergraphs and six values of k , a single pass computes the best streaming partitions in geometric mean for both objectives, with connectivity better than `FREIGHT`’s by a factor of 1.51 and cut-net by 1.15 at 2.9 to 3.1 times `FREIGHT`’s time per pin (72–75 ns), so that the streaming Pareto frontier of quality against time consists of `Hashing`, `FREIGHT` and HedgeStream; the connectivity lead persists in extended-grid experiments up to $k = 131\,072$, and three restreaming passes buy another 2.7% connectivity and 4.7% cut-net at twice the time and identical memory. On average HedgeStream also outperforms two in-memory partitioners at a fraction of their time and memory: `HYPE` on both objectives, and `SIGHP` in connectivity, driven by the sparse-matrix family, whereas `PaToH` remains 2.6 times better in connectivity at 21 times the time: the gap to `PaToH` is what a single pass costs, and the SAT instances, on which `SIGHP` stays ahead, mark the limit of the per-vertex sketch, which larger capacities do not lift.

In the true streaming setting, the whole benchmark read from disk returns exactly the in-memory partitions, at no more than 506 MiB of memory for HedgeStream and 175 MiB for **FREIGHT**; over the five largest instances a pass takes 1.1 to 3.5 times **FREIGHT**’s warm-cache wall-clock time with the readers as shipped, and **FREIGHT**’s per-net state stays the leaner one until nets outnumber vertices, where the ordering flips (with twelve nets per vertex, on `sat_nurikabe_p18`, HedgeStream streams in half of **FREIGHT**’s memory). The vertex-stream model moreover needs the input transposed into a second on-disk copy before it can start, which the hyperedge-stream model does not; an even fairer comparison would charge **FREIGHT** a streaming transposition, but charged or not, the two tools operate in different streaming models.

A natural next step is shared-memory parallelism: the pass is sequential by definition, but hyperedges without common pins could be processed concurrently, the sketch updates are per-vertex and amenable to atomic or per-vertex-lock protection, and the buffer flush already materializes a batch of nets that threads could share; the incremental bucket queue itself partitions naturally among workers, one disjoint range of buckets per thread, so that draining is contention-free and only a net’s promotion into a fuller bucket crosses thread boundaries; and since Misra–Gries sketches merge, threads could collect their updates in thread-local partial sketches and fold them into the global per-vertex array at flush boundaries, trading a slightly stale affinity signal for uncontended writes; what concurrency costs in determinism, and in the quality of a load penalty that would then read shifting block sizes, is open. HedgeStream will be available as part of **CHSZLabLib** and in its own GitHub repository; the experimental data, the baseline sources, the format converters and the run scripts of this evaluation will be released with it.

AI Disclosure. Claude Opus 5 was used for writing, editing and code assistance.

Acknowledgements. Funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – project number 519626652.

References

- [1] Yaroslav Akhremtsev, Peter Sanders, and Christian Schulz. High-quality shared-memory graph partitioning. In *Euro-Par 2018: Parallel Processing - 24th International Conference on Parallel and Distributed Computing, Turin, Italy, August 27-31, 2018, Proceedings*, volume 11014 of *Lecture Notes in Computer Science*, pages 659–671, 2018. doi:10.1007/978-3-319-96983-1_47.
- [2] Dan Alistarh, Jennifer Iglesias, and Milan Vojnovic. Streaming min-max hypergraph partitioning. In *Advances in Neural Information Processing Systems*, pages 1900–1908, 2015.
- [3] Charles J. Alpert. The ISPD98 circuit benchmark suite. In Majid Sarrafzadeh, editor, *Proceedings of the 1998 International Symposium on Physical Design, ISPD 1998, Monterey, CA, USA, April 6-8, 1998*, pages 80–85, 1998. doi:10.1145/274535.274546.
- [4] Amel Awadelkarim and Johan Ugander. Prioritized restreaming algorithms for balanced graph partitioning. In *KDD ’20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23-27, 2020*, pages 1877–1887, 2020. doi:10.1145/3394486.3403239.

- [5] Linus Baumgärtner, Adil Chhabra, Marcelo Fonseca Faraj, and Christian Schulz. BuffCut: Prioritized buffered streaming graph partitioning. In *24th International Symposium on Experimental Algorithms, SEA 2026*, volume 371 of *LIPIcs*, pages 5:1–5:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026. doi:10.4230/LIPIcs.SEA.2026.5.
- [6] Anton Belov, Daniel Diepold, Marijn Heule, and Matti Järvisalo. Proceedings of SAT Competition 2014: Solver and benchmark descriptions. University of Helsinki, Department of Computer Science, Series of Publications B, vol. B-2014-2, <https://helda.helsinki.fi/handle/10138/135571>, 2014.
- [7] Radu Berinde, Piotr Indyk, Graham Cormode, and Martin J. Strauss. Space-optimal heavy hitters with strong error bounds. *ACM Transactions on Database Systems*, 35(4):26:1–26:28, 2010. doi:10.1145/1862919.1862923.
- [8] Ruchi Bhoot, Tuhin Khare, Manoj Agarwal, Siddharth D. Jaiswal, and Yogesh Simmhan. Triparts: Scalable streaming graph partitioning to enhance community structure. *Proceedings of the VLDB Endowment*, 18(9):2992–3006, 2025. doi:10.14778/3746405.3746423.
- [9] Ulrik Brandes, Daniel Delling, Marco Gaertler, Robert Görke, Martin Hoefer, Zoran Nikoloski, and Dorothea Wagner. On modularity clustering. *IEEE transactions on knowledge and data engineering*, 20(2):172–188, 2008. doi:10.1109/TKDE.2007.190689.
- [10] Aydın Buluç, Henning Meyerhenke, Ilya Safro, Peter Sanders, and Christian Schulz. Recent advances in graph partitioning. In *Algorithm Engineering: Selected Results and Surveys*, volume 9220 of *Lecture Notes in Computer Science*, pages 117–158. Springer International Publishing, Cham, 2016. doi:10.1007/978-3-319-49487-6_4.
- [11] Ümit V. Çatalyürek and Cevdet Aykanat. PaToH (Partitioning Tool for Hypergraphs). In David A. Padua, editor, *Encyclopedia of Parallel Computing*, pages 1479–1487. 2011. doi:10.1007/978-0-387-09766-4_93.
- [12] Ümit V. Çatalyürek, Karen D. Devine, Marcelo Fonseca Faraj, Lars Gottesbüren, Tobias Heuer, Henning Meyerhenke, Peter Sanders, Sebastian Schlag, Christian Schulz, Daniel Seemaier, and Dorothea Wagner. More recent advances in (hyper)graph partitioning. *ACM Computing Surveys*, 55(12):253:1–253:38, 2023. doi:10.1145/3571808.
- [13] Adil Chhabra, Marcelo Fonseca Faraj, Christian Schulz, and Daniel Seemaier. Buffered streaming edge partitioning. In *22nd International Symposium on Experimental Algorithms, SEA 2024*, volume 301 of *LIPIcs*, pages 5:1–5:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024. doi:10.4230/LIPIcs.SEA.2024.5.
- [14] Timothy A. Davis and Yifan Hu. The University of Florida sparse matrix collection. *ACM Transactions on Mathematical Software*, 38(1):1:1–1:25, 2011. doi:10.1145/2049662.2049663.
- [15] Zezhong Ding, Deyu Kong, Zhuoxu Zhang, Xike Xie, and Jianliang Xu. ClusPar: A game-theoretic approach for efficient and scalable streaming edge partitioning. *IEEE Transactions on Computers*, 74(1):116–130, 2025. doi:10.1109/TC.2024.3475568.

- [16] Zezhong Ding, Yongan Xiang, Shangyou Wang, Xike Xie, and S. Kevin Zhou. Play like a vertex: A Stackelberg game approach for streaming graph partitioning. *Proceedings of the ACM on Management of Data*, 2(3):162, 2024. doi:10.1145/3654965.
- [17] Kamal Eyubov, Marcelo Fonseca Faraj, and Christian Schulz. FREIGHT: Fast streaming hypergraph partitioning. *Algorithmica*, 87(3):405–428, 2025. doi : 10 . 1007 / s00453-024-01291-8.
- [18] Marcelo Fonseca Faraj and Christian Schulz. Buffered streaming graph partitioning. *ACM J. Exp. Algorithmics*, 27, 2022. doi:10.1145/3546911.
- [19] Marcelo Fonseca Faraj and Christian Schulz. Recursive multi-section on the fly: Shared-memory streaming algorithms for hierarchical graph partitioning and process mapping. In *2022 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 473–483, 2022. doi:10.1109/CLUSTER51413.2022.00057.
- [20] Marcelo Fonseca Faraj, Alexander van der Grinten, Henning Meyerhenke, Jesper Larsson Träff, and Christian Schulz. High-quality hierarchical process mapping. In *18th International Symposium on Experimental Algorithms, SEA*, volume 160 of *LIPIcs*, pages 4:1–4:15, 2020. doi:10.4230/LIPIcs.SEA.2020.4.
- [21] Lars Gottesbüren, Tobias Heuer, Nikolai Maas, Peter Sanders, and Sebastian Schlag. Scalable high-quality hypergraph partitioning. *ACM Transactions on Algorithms*, 20(1):9:1–9:54, 2024. doi:10.1145/3626527.
- [22] Lars Gottesbüren, Tobias Heuer, Peter Sanders, Christian Schulz, and Daniel Seemaier. Deep multilevel graph partitioning. In *29th Annual European Symposium on Algorithms, ESA*, volume 204 of *LIPIcs*, pages 48:1–48:17, 2021. doi:10.4230/LIPIcs.ESA.2021.48.
- [23] Huhao Guan, Zezhong Ding, Ao Ke, Xike Xie, and S. Kevin Zhou. SIGHP: Scalable information-guided hypergraph partitioner. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pages 336–347. ACM, 2026. doi:10.1145/3770854.3780266.
- [24] Milad Rezaei Hajidehi, Sraavan Sridhar, and Margo I. Seltzer. CUTTANA: Scalable graph partitioning for faster distributed graph databases and analytics. *Proceedings of the VLDB Endowment*, 18(1):14–27, 2024. doi:10.14778/3696435.3696437.
- [25] Barbara Hoffmann, Shai Dorian Peretz, Adil Chhabra, Ahmet Kadir Yalcinkaya, Ruben Mayer, and Christian Schulz. SIGMA: A versatile streaming graph partitioner for vertex- and edge-balanced distributed GNN training. arXiv preprint arXiv:2606.03519, 2026. doi:10.48550/arXiv.2606.03519.
- [26] Nazanin Jafari, Oguz Selvitopi, and Cevdet Aykanat. Fast shared-memory streaming multilevel graph partitioning. *Journal of Parallel and Distributed Computing*, 147:140–151, 2021. doi : 10.1016/j.jpdc.2020.09.004.
- [27] George Karypis and Vipin Kumar. Parallel multilevel k-way partitioning scheme for irregular graphs. In *Proceedings of the ACM/IEEE Conference on Supercomputing*, page 35, 1996. doi:10.1109/SC.1996.32.

- [28] George Karypis and Vipin Kumar. Multilevel k-way hypergraph partitioning. *VLSI Design*, 11(3):285–300, 2000. doi:10.1155/2000/19436.
- [29] Christian Mayer, Ruben Mayer, Sukanya Bhowmik, Lukas Epple, and Kurt Rothermel. HYPE: massive hypergraph partitioning with neighborhood expansion. In *IEEE International Conference on Big Data (IEEE BigData)*, pages 458–467, 2018. doi:10.1109/BigData.2018.8621968.
- [30] Ruben Mayer, Kamil Orujzade, and Hans-Arno Jacobsen. Out-of-core edge partitioning at linear run-time. In *38th IEEE International Conference on Data Engineering, ICDE 2022*, pages 2629–2642. IEEE, 2022. doi:10.1109/ICDE53745.2022.00242.
- [31] Lingkai Meng, Long Yuan, Xuemin Lin, Wenjie Zhang, and Ying Zhang. Triangle counting in hypergraph streams: A complete and practical approach. *Proceedings of the ACM on Management of Data*, 3(6):372:1–372:28, 2025. doi:10.1145/3769837.
- [32] Jayadev Misra and David Gries. Finding repeated elements. *Science of Computer Programming*, 2(2):143–152, 1982. doi:10.1016/0167-6423(82)90012-0.
- [33] Carlos Fernandez Musoles, Daniel Coca, and Paul Richmond. HyperPRAW: Architecture-aware hypergraph restreaming partition to improve performance of parallel applications running on high performance computing systems. In *Proceedings of the 48th International Conference on Parallel Processing, ICPP 2019*, pages 19:1–19:10. ACM, 2019. doi:10.1145/3337821.3337876.
- [34] Joel Nishimura and Johan Ugander. Restreaming graph partitioning: simple versatile algorithms for advanced balancing. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1106–1114, 2013. doi:10.1145/2487575.2487696.
- [35] Md Anwarul Kaium Patwary, Saurabh Kumar Garg, and Byeong Kang. Window-based streaming graph partitioning algorithm. In *Proceedings of the Australasian Computer Science Week Multiconference, ACSW*, pages 51:1–51:10, 2019. doi:10.1145/3290688.3290711.
- [36] François Pellegrini and Jean Roman. SCOTCH: A software package for static mapping by dual recursive bipartitioning of process and architecture graphs. In *High-Performance Computing and Networking*, volume 1067 of *Lecture Notes in Computer Science*, pages 493–498. Springer, 1996. doi:10.1007/3-540-61142-8_588.
- [37] Maria Predari, Charilaos Tzovas, Christian Schulz, and Henning Meyerhenke. An MPI-based algorithm for mapping complex networks onto hierarchical architectures. In *Euro-Par 2021: Parallel Processing - 27th International Conference on Parallel and Distributed Computing*, volume 12820 of *LNCS*, pages 167–182, 2021. doi:10.1007/978-3-030-85665-6_11.
- [38] Henrik Reinstädter, S. M. Ferdous, Alex Pothen, Bora Uçar, and Christian Schulz. Semi-streaming algorithms for hypergraph matching. In *33rd Annual European Symposium on Algorithms, ESA 2025*, volume 351 of *LIPIcs*, pages 79:1–79:19. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. doi:10.4230/LIPIcs.ESA.2025.79.

- [39] Peter Sanders and Christian Schulz. Think locally, act globally: Highly balanced graph partitioning. In *Experimental Algorithms, 12th International Symposium, SEA*, volume 7933 of *LNCS*, pages 164–175, 2013. doi:10.1007/978-3-642-38527-8_16.
- [40] Sebastian Schlag, Vitali Henne, Tobias Heuer, Henning Meyerhenke, Peter Sanders, and Christian Schulz. k -way hypergraph partitioning via n -level recursive bisection. In *Proceedings of the Eighteenth Workshop on Algorithm Engineering and Experiments, ALENEX*, pages 53–67, 2016. doi:10.1137/1.9781611974317.5.
- [41] Sebastian Schlag, Tobias Heuer, Lars Gottesbüren, Yaroslav Akhremtsev, Christian Schulz, and Peter Sanders. High-quality hypergraph partitioning. *ACM Journal of Experimental Algorithmics*, 27:1.9:1–1.9:39, 2022. doi:10.1145/3529090.
- [42] Christian Schulz and Darren Strash. Graph partitioning: Formulations and applications to big data. In *Encyclopedia of Big Data Technologies*. 2019. doi:10.1007/978-3-319-63962-8_312-2.
- [43] Isabelle Stanton and Gabriel Kliot. Streaming graph partitioning for large distributed graphs. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1222–1230, 2012. doi:10.1145/2339530.2339722.
- [44] Fatih Taşyaran, Berkay Demireller, Kamer Kaya, and Bora Uçar. Streaming Hypergraph Partitioning Algorithms on Limited Memory Environments. In *HPCS 2020 - International Conference on High Performance Computing & Simulation*, pages 1–8. IEEE, 2021. URL: <https://hal.archives-ouvertes.fr/hal-03182122>.
- [45] Charalampos Tsourakakis, Christos Gkantsidis, Bozidar Radunovic, and Milan Vojnovic. Fennel: Streaming graph partitioning for massive scale graphs. In *Proceedings of the 7th ACM international conference on Web search and data mining*, pages 333–342, 2014. doi:10.1145/2556195.2556213.